

Apuntes Completos de Programación Avanzada

TEMA 1 - Introducción al Desarrollo de Aplicaciones con Frameworks Modernos

Propósito de esta guía

Estos apuntes están diseñados como una guía de estudio avanzada. No solo memorizarás, sino que comprenderás la arquitectura moderna, blindando tu conocimiento técnico contra preguntas complejas o "trampa" de examen.

Índice del Tema

- [1. Arquitecturas Típicas de Aplicaciones](#)
- [2. Entorno de Desarrollo y Persistencia \(JPA + JDBC\)](#)
- [3. Visión Global del Backend \(La Capa Modelo y Servicios\)](#)
- [4. Visión Global del Frontend con React](#)

1. Arquitecturas Típicas de Aplicaciones

1.1 Intuición y Estructura Formal

La "arquitectura" distribuye las responsabilidades entre el cliente (navegador) y el servidor, definiendo cómo se comunican. El documento establece una arquitectura de referencia estricta:

- **Frontend (Lado del Cliente):**
 - **Capa IU (Interfaz de Usuario):** Pinta la información y reacciona al usuario.
 - **Capa Acceso a Servicios:** Se comunica por la red con el servidor.
- **Backend (Lado del Servidor):**
 - **Capa Servicios:** Expone la funcionalidad al exterior (la "puerta" del servidor).
 - **Capa Modelo:** ¡Concepto Crítico! Esta gran capa agrupa internamente dos subcapas fundamentales:
 - **Capa Lógica de Negocio:** El cerebro y las reglas de la aplicación.
 - **Capa Acceso a Datos:** Se comunica con la Base de Datos (BD).

 **Además de las aplicaciones nativas y SPA, existe la arquitectura de *Aplicación web del lado servidor con capa Modelo remota*, en la cual el Servidor de Aplicaciones y el Backend no residen en la misma máquina, comunicándose a través de la red mediante un Servicio Web.**

1.2 La Arquitectura SPA y su Ecosistema

En una aplicación **SPA** (*Single Page Application*), el servidor web devuelve un único fichero HTML inicial (`index.html`) muy ligero junto con el código JavaScript. A partir de ahí, el JavaScript toma el control total en el navegador, manipulando el DOM dinámicamente y haciendo peticiones asíncronas al Backend.

Alternativas SPA en la Industria

Aunque en esta asignatura usaremos **React**, el ecosistema moderno de frontends SPA está dominado por tres grandes rivales que debes conocer: **React, Angular y Vue.js**.

1.3 Bloque de Repaso: Sección 1

Resumen Rápido

Pasamos de webs tradicionales renderizadas en servidor a SPAs donde el frontend es rico e independiente. El backend moderno se divide formalmente en la Capa de Servicios y la Capa Modelo (que integra Lógica y Datos).

- **Ideas Clave:** En SPA, el intercambio de información entre Frontend y Backend se hace puramente en formato **JSON** vía peticiones asíncronas.
- **Conceptos a Memorizar (Estructura Formal):** Capa Modelo = Capa Lógica de Negocio + Capa Acceso a Datos .
- **Errores Comunes:** Separar la Lógica de Negocio y el Acceso a Datos como capas inconexas. Ambas forman la Capa Modelo.

2. Entorno de Desarrollo y Persistencia (JPA + JDBC)

Para materializar esta arquitectura en nuestro caso de estudio (*PA Shop*), utilizamos el siguiente *stack*:

- **Frontend SPA:** React y su ecosistema de librerías.
- **Backend Java:** Ecosistema Spring.
- **Persistencia de Datos (El gran detalle técnico):** Para la Capa de Acceso a Datos usamos **JPA** (*Java Persistence API*), que es el estándar de mapeo objeto-relacional en Java.

Detalle Técnico de Examen

JPA es solo una API (una especificación). Implementaciones de JPA como *Hibernate* son las que hacen el trabajo duro. Lo crucial es saber que, internamente, **todas las implementaciones de JPA hacen uso de la API JDBC** para hablar finalmente con la base de datos relacional.

3. Visión Global del Backend (La Capa Modelo y Servicios)

Analicemos cómo se construye el backend en Spring.

3.1 Capa de Acceso a Datos (Parte de la Capa Modelo)

Aquí las tablas se modelan como **Entidades Java** (ej. `Order` , `User`). Las relaciones entre tablas no se manejan con simples claves foráneas (`Long userId`), sino de manera natural y orientada a objetos, usando referencias a las propias clases (`User user`).

DAOs y Paginación (Crítico para escalabilidad):

Para consultar la BD, creamos interfaces DAO. Spring Data genera las consultas automáticamente a partir del nombre del método (ej. `findByUserIdOrderByDateDesc`).

Sin embargo, si un usuario tiene miles de pedidos, cargar todos de golpe con un simple `List<Order>` saturaría la memoria del servidor. Para evitar esto, **exigimos paginación**. Los DAOs profesionales deben extender de `PagingAndSortingRepository` (que a su vez extiende de `CrudRepository` y añade capacidades de paginado) y utilizar firmas que devuelvan porciones de datos:

```
public interface OrderDao extends PagingAndSortingRepository<Order, Long> {
    Slice<Order> findByUserIdOrderByDateDesc(Long userId, Pageable pageable);
}
```

- `Pageable` : Objeto que indica a Spring qué "página" de datos queremos y de qué tamaño.
- `Slice` : Objeto devuelto que contiene solo ese bloque de datos y un *flag* (`hasNext`) que indica si hay más.

📌 3.2 Capa de Lógica de Negocio (Parte de la Capa Modelo)

Aquí residen los servicios (`ShoppingServiceImpl`) que orquestan las operaciones complejas.

Transaccionalidad Declarativa (`@Transactional`):

Agrupar múltiples operaciones de base de datos para que ocurran de forma atómica (o todas, o ninguna). En Spring se declara usando `@Transactional`.

$$\text{Resultado Transacción} = \begin{cases} \text{Commit (Guardar)}, & \text{si } E = \emptyset \vee E \in \text{CheckedException} \\ \text{Rollback (Deshacer)}, & \text{si } E \in \text{RuntimeException} \vee E \in \text{Error} \end{cases}$$

- **Intuición:** Las transacciones protegen la integridad. Si hay un fallo de sistema imprevisto (`RuntimeException`), se deshace todo por seguridad. Si lanzamos un error previsible, de negocio, validado por nosotros (`CheckedException`), los cambios hechos hasta ese punto sí se guardan (*Commit*).

📌 3.3 Capa de Servicios REST (La Frontera)

Es la puerta de entrada HTTP. Los Controladores (ej. `ShoppingController`) reciben peticiones en rutas (`/shopping/orders/{orderId}`). Extraen variables con anotaciones como `@PathVariable` y Spring convierte mágicamente las respuestas de objetos DTO (*Data Transfer Objects*) a JSON.

📌 3.4 Bloque de Repaso: Sección 3

📖 Resumen Rápido

La Capa Modelo domina el backend usando entidades mapeadas a objetos y DAOs paginados (`Slice` , `Pageable`) para no saturar memoria. La Lógica de Negocio se protege con un estricto control transaccional matemático, y la Capa de Servicios traduce este mundo a REST/JSON.

- **Ideas Clave:** Las entidades se relacionan referenciando objetos enteros, no IDs sueltos. La paginación previene los cuellos de botella de memoria en consultas masivas.
- **Conceptos a Memorizar:**
 - Fórmula Transaccional: Rollback SOLO en `RuntimeException` o `Error`.
 - Paginación API: `PagingAndSortingRepository` , `Slice<T>` , `Pageable`.
- **Errores Comunes (Pregunta Trampa):** Creer que un DAO paginado carga todo en memoria y luego lo corta. **Falso:** La paginación usa internamente la API de JDBC para pedir a la base de datos *estrictamente* el rango de resultados solicitado, optimizando la red y la memoria.

4. Visión Global del Frontend con React

4.1 Comunicación con el Backend

La Capa de Acceso a Servicios utiliza código asíncrono para consumir la API. Definimos funciones que hacen el *fetch* al servidor, esperan el JSON (`await`), y actualizan la interfaz solo con los datos (*payload*).

4.2 Interfaz Basada en Componentes

La Capa IU se diseña como un puzzle usando React. Un componente es una porción autónoma de interfaz.

- **Divide y vencerás:** Una página gigante se rompe en trozos manejables (Ej: `App` → `Header` → `ShoppingCartIcon`).
- **Reusabilidad:** Si un componente como `ShoppingItemList` está bien programado, te servirá en 5 vistas diferentes sin reprogramarlo.

Veredicto del Tutor para el Examen

Con esta versión actualizada de los apuntes, dominas el "Bosque" del Tema 1 con una precisión de francotirador. Entiendes que la Capa Modelo no es un ente abstracto, sino la unión de tu Lógica y tus Datos. Sabes que tus Datos se protegen con paginación (`slice`) en los DAOs para que la aplicación sea escalable, y dominas la regla transaccional al dedillo. ¡Estás listo para arrasar en cualquier test teórico sobre este tema!

Tema 2 - Capa de Acceso a Datos con Spring y JPA

📌 Propósito de esta guía

Esta versión unificada ha sido refinada para alcanzar el 100% de cobertura técnica del temario y protegerte contra las trampas sintácticas más crueles de los test oficiales. Recuerda: el objetivo de la asignatura no es memorizar fragmentos aislados de código, sino comprender el porqué arquitectónico de cada decisión para saber aplicarlo con precisión quirúrgica a cualquier dominio de negocio nuevo.

🗺 Índice del Tema

- [1. El Ecosistema de Persistencia: JPA, Hibernate, JDBC y Spring Data JPA](#)
- [2. Modelado Básico de Entidades Java](#)
- [3. Modelado de Relaciones Bidireccionales \(Y Trampas de Examen\)](#)
- [4. Estrategias de Carga: LAZY vs. EAGER y la Mecánica de Proxies](#)
- [5. El Patrón Domain Model y la anotación @Transient](#)
- [6. Desarrollo de DAOs Eficientes con Spring Data JPA](#)
- [7. Configuración de Entornos con YAML](#)
- [8. Bloque de Evaluación Forense y Simulación de Examen](#)

1. El Ecosistema de Persistencia: JPA, Hibernate, JDBC y Spring Data JPA

Antes de escribir código, debemos entender de forma rigurosa la cadena de mando tecnológica que conecta nuestra aplicación Java orientada a objetos con un motor de Base de Datos Relacional (como MySQL o Oracle).

- **JDBC (Java Database Connectivity):** Es la API estándar de más bajo nivel de Java para interactuar con sistemas de gestión de bases de datos (*SGDb*). Trabajar con JDBC puro exige programar sentencias SQL embebidas en cadenas de texto (*strings*) gigantes, abrir/cerrar conexiones manualmente y mapear de forma artesanal cada columna a atributos de objetos. Es un enfoque tedioso, propenso a errores de sintaxis y acoplado al motor de base de datos.
- **JPA (Java Persistence API / Jakarta Persistence):** Para mitigar las deficiencias de JDBC, irrumpe el paradigma **ORM** (*Object-Relational Mapping*). JPA **no es una librería ni un framework ejecutable**; es exclusivamente una especificación estándar de Java. Define una interfaz unificada y un catálogo de anotaciones estándar (`@Entity`, `@Id`, `@Column`) que dictaminan formalmente cómo traducir clases Java a tablas SQL.
- **Hibernate:** Es la implementación líder del estándar JPA del mercado que realiza el "trabajo sucio". Hibernate toma tus clases anotadas con JPA, interpreta el dialecto de la base de datos configurada, genera automáticamente el código SQL correspondiente en tiempo de ejecución y lo ejecuta haciendo uso de la API interna de **JDBC** por debajo.
- **Spring Data JPA:** Es la capa superior de abstracción de Spring. Elimina por completo el código repetitivo (*boilerplate*) al crear componentes de acceso a datos o **DAOs** (*Data Access Objects*). En lugar de codificar clases de persistencia, Spring Data te permite declarar simples interfaces Java y se encarga de inyectar dinámicamente las implementaciones y consultas optimizadas por debajo.

📋 Intuición Estricta del Flujo de Ejecución

Tu Código → Interfaz Spring Data → Hibernate → Anotaciones JPA → SQL vía JDBC → Base de Datos

2. Modelado Básico de Entidades Java

Una **Entidad** representa una tabla de la base de datos mapeada bajo el paradigma ORM. Cambiamos la mentalidad relacional de "filas y columnas" por una mentalidad puramente orientada a objetos de "objetos y atributos".

2.1 Requisitos Formales de una Entidad

Para que el motor de persistencia de JPA acepte y gestione una clase Java como una entidad válida, esta debe cumplir de forma irrenunciable con **cuatro reglas estrictas**:

1. Estar decorada obligatoriamente con la anotación `@Entity` a nivel de clase.
2. Contar con un **constructor sin argumentos** (vacío), que puede tener visibilidad `public` o `protected` (requerido por Hibernate para instanciar el objeto mediante reflexión).
3. La clase **no puede ser final** (`public final class`), ya que JPA necesita heredar de ella para crear objetos intermediarios (*proxies*).
4. Poseer obligatoriamente un atributo o propiedad marcado explícitamente como Clave Primaria mediante la anotación `@Id`.

2.2 Estrategia de Acceso al Estado

JPA necesita definir el mecanismo mediante el cual leerá y escribirá valores en los atributos de tus clases. Existen dos estrategias de acceso mutuamente excluyentes determinadas por la ubicación física de la anotación `@Id`:

- **Acceso por campo:** Si la anotación `@Id` se sitúa directamente encima de la variable privada (`private Long id;`), JPA ignorará tus métodos getters/setters y accederá de forma directa al campo por reflexión.
- **Acceso por propiedad (Enfoque Oficial de la Asignatura):** Si la anotación `@Id` se sitúa encima del método *getter* correspondiente (`public Long getId()`), JPA asume la estrategia de propiedad. **Consecuencia de Examen:** Todas las demás anotaciones de mapeo relacional (`@Column`, `@ManyToOne`, `@JoinColumn`, etc.) deben ubicarse **SIEMPRE encima de sus respectivos métodos get()** (o `is()` si el atributo es booleano), nunca sobre los campos privados.

2.3 Claves Primarias y Mapeo de Tipos Nativos

Para delegar la asignación de claves primarias al motor de base de datos, acompañamos el `@Id` con `@GeneratedValue`:

Estrategia ID = $\begin{cases} \text{GenerationType.IDENTITY,} & \text{Para columnas auto-incrementales (Típico en MySQL).} \\ \text{GenerationType.SEQUENCE,} & \text{Usa secuencias nativas de BD (Típico en Oracle), requiere @SequenceC} \end{cases}$

Mapeo Automatizado de Tipos:

JPA realiza una equivalencia transparente entre tipos nativos de Java y tipos SQL:

- Tipos primitivos, clases envolventes (`Long`, `Integer`), `String` y `BigDecimal`.
 - Tipos de fecha modernos de la API `java.time`: `LocalDateTime`, `LocalDate`.
 - **Advertencia con los Enumerados (`enum`):** Por defecto, JPA persiste los enumerados en la BD usando su valor posicional numérico (*Ordinal int*). Si reorganizas tu código cambiando el orden del `enum`, corromperás toda la consistencia de tu base de datos histórica.
-

3. Modelado de Relaciones Bidireccionales (Y Trampas de Examen)

En diseño relacional, las tablas se vinculan de forma unívoca inyectando una clave foránea (*FK*) numérica en la tabla secundaria. En el paradigma ORM, modelamos las relaciones de forma natural mediante **referencias cruzadas a objetos de clases enteras**.

En una relación **bidireccional** (aquella que permite navegar en ambos sentidos), es obligatorio parametrizar y distinguir dos roles fundamentales:

📌 3.1 El Lado Propietario (Owning Side)

Es el componente que posee la responsabilidad de la relación. En términos de base de datos, mapea directamente a la tabla física que **contiene físicamente la columna con la Clave Foránea (*FK*)**.

- En una relación un-pedido-a-muchas-líneas (1:N), **el lado propietario es siempre el lado N** (el hijo, ej. la Línea de Pedido).
- Se decora encima del método *getter* con la anotación `@ManyToOne`.
- Se acompaña obligatoriamente de `@JoinColumn(name = "nombre_columna_fk")` para indicar formalmente el nombre exacto que tendrá la columna de clave foránea en la tabla SQL.

📌 3.2 El Lado Inverso (Inverse Side)

Actúa puramente como el "espejo" o reflejo de la relación para permitir la navegación inversa. Su tabla correspondiente en base de datos **NO almacena ninguna clave foránea**.

- En una relación 1:N, el lado inverso es siempre el **lado 1** (el padre, ej. el Pedido).
- Se decora encima de su método *getter* que devuelve una colección (preferiblemente un `Set<T>` para evitar duplicidades) con `@OneToMany`.
- **Requisito Sintáctico Ineludible:** Debe incluir obligatoriamente el atributo `mappedBy = "nombreAtributoLadoPropietario"`. Esto le comunica a JPA: *"Yo no gobierno esta relación ni tengo la FK; busca las instrucciones de mapeo inspeccionando esta propiedad concreta dentro de la clase hija"*.

📌 3.3 Ejemplo Resuelto Estándar de Mapeo Relacional

Analicemos la relación entre `Order` (Pedido) y `OrderItem` (Línea de Pedido):

```
// DENTRO DE LA CLASE ENTIDAD: OrderItem (Lado N - Lado Propietario)
@ManyToOne(optional = false, fetch = FetchType.LAZY)
@JoinColumn(name = "orderId") // Crea físicamente la columna orderId como FK en MySQL
public Order getOrder() {
    return this.order;
}

// DENTRO DE LA CLASE ENTIDAD: Order (Lado 1 - Lado Inverso)
@OneToMany(mappedBy = "order") // Apunta al atributo/getter "order" de OrderItem
public Set<OrderItem> getOrderItems() {
    return this.orderItems;
}
```

🔗 Control de Cardinalidad Mínima

Configurar `optional = false` dentro de la anotación `@ManyToOne` valida e impone a nivel de arquitectura y esquema que una entidad `OrderItem` **jamás** podrá existir en el sistema de manera huérfana sin un `Order` padre asociado (Fija la cardinalidad mínima estrictamente en 1).

⚡ ¡TRAMPA MORTAL DE EXAMEN! El Error del mappedBy

En los cuestionarios tipo test oficiales, es extremadamente común intentar engañar al alumno presentando opciones con el atributo `mappedBy` ubicado dentro de una anotación `@ManyToOne`.

Regla de oro absoluta de examen: La anotación `@ManyToOne` **NUNCA** soporta ni tiene el atributo `mappedBy`. Si detectas código que declare `@ManyToOne(mappedBy = "...")`, es un **error de compilación y sintaxis directo** que invalida automáticamente esa opción. El atributo `mappedBy` es de uso exclusivo del lado inverso (`@OneToMany`, `@OneToOne`).

4. Estrategias de Carga: LAZY vs. EAGER y la Mecánica de Proxies

¿Qué ocurre internamente cuando solicitas a Spring Data cargar un objeto `Order` de la base de datos? ¿Debe el framework lanzar consultas adicionales para descargar de forma simultánea todas las líneas de pedido vinculadas en ese mismo instante? La respuesta depende de la política de carga (*Fetching Strategy*).

- **EAGER (Ansiosa):** Fuerza la recuperación inmediata de la entidad principal y de todos sus objetos relacionados. **Es la política por defecto en JPA para relaciones de cardinalidad simple (`@ManyToOne` y `@OneToOne`)**. Constituye un peligro crítico para el rendimiento del servidor backend. Si recuperas una simple línea de pedido, una cascadeada EAGER podría arrastrar el pedido entero, el cual a su vez arrastraría al usuario creador, cargando megabytes de datos innecesarios en la memoria RAM y saturando la red.
- **LAZY (Perezosa - Enfoque Recomendado):** Retarda la carga de las relaciones de forma perezosa bajo demanda estricta. **Es la política por defecto en relaciones de colección (`@OneToMany` y `@ManyToMany`)**. Al solicitar la entidad principal, las propiedades asociadas no se descargan de la BD; en su lugar, se inyectan componentes especiales llamados **Proxies**.

🔪 Comportamiento Interno y Magia de los Proxies

Un **Proxy** es un objeto "falso" generado dinámicamente en tiempo de ejecución por Hibernate que suplanta la identidad de tu entidad destino. Este objeto instrumental está inicialmente vacío y **solo conoce el valor de la Clave Primaria (ID)** del registro de la base de datos.

💡 La Analogía del Cheque en Blanco

Un proxy configurado como **LAZY** funciona exactamente igual que tener un cheque en blanco en el bolsillo (RAM). El objeto ocupa un espacio referencial, pero los fondos (los datos reales de la BD) siguen custodiados en el banco (MySQL). Solo en el instante preciso en el que requieres hacer uso del valor, ejecutas el proceso de acudir al banco a por el dinero líquido.

Leyes de Activación de un Proxy LAZY (Pregunta Fija de Examen):

1. Si tu código invoca al método `proxy.getId()`, **NO se genera ninguna consulta SQL contra la base de datos**, puesto que el proxy ya dispone de la clave primaria en su estructura interna desde su inicialización.
2. Si invocas **CUALQUIER otro método** de la interfaz pública del objeto (`proxy.getName()`, `proxy.getPrice()`), el proxy detiene transparentemente el hilo de ejecución de la aplicación, dispara una sentencia `SELECT` dirigida a la base de datos para rellenar sus campos vacíos (mecanismo de *inicialización del proxy*), y retorna el dato solicitado.
3. Si el proxy envuelve una colección (`Set<OrderItem>`), la sentencia `SELECT` complementaria se disparará únicamente en el instante en que ejecutes una consulta operativa sobre su tamaño (`items.size()`), invoques un método de extracción (`items.contains(...)`) o intentes iterarla mediante un bucle `for` o un flujo secuencial.

5. El Patrón Domain Model y la anotación @Transient

El diseño de arquitecturas orientadas a objetos modernas rechaza el antipatrón de **Modelo de Dominio Anémico** (entidades que actúan como meras cajas tontas de datos con getters/setters sin inteligencia). Se promueve el patrón **Domain Model**, donde las entidades encapsulan la lógica de negocio intrínseca estrechamente ligada a su estado.

- *Ejemplo:* Calcular el importe acumulado total de un pedido no es responsabilidad de un servicio externo, sino un comportamiento propio que debe resolver la propia entidad `Order` iterando sobre sus líneas.

```
// LÓGICA DE NEGOCIO INYECTADA EN LA ENTIDAD: Order
private Set<OrderItem> orderItems;

@Transient // ¡Obligatorio! Evita que JPA busque una columna "totalPrice" en SQL
public BigDecimal getTotalPrice() {
    BigDecimal total = BigDecimal.ZERO;
    for (OrderItem item : this.orderItems) {
        total = total.add(item.getSubtotal());
    }
    return total;
}
```

🔥 El Conflicto de Nomenclatura con Acceso por Propiedad

Si tu arquitectura implementa la estrategia de **Acceso por propiedad**, JPA inspecciona los métodos cuyo prefijo comience por `get` o `is` para deducir las columnas de las tablas. Si creas el método de negocio `getTotalPrice()`, JPA asumirá erróneamente que la tabla de la base de datos posee físicamente una columna llamada `totalPrice`. Al no existir, el sistema lanzará una excepción catastrófica en el arranque.

La Solución Obligatoria: Decorar el método de negocio con la anotación `@Transient`. Esto le ordena taxativamente al motor de JPA: *"Ignora este método getter por completo; constituye una función matemática pura de negocio y no guarda ninguna correspondencia con el esquema relacional físico"*.

6. Desarrollo de DAOs Eficientes con Spring Data JPA

La persistencia operativa real (ejecutar sentencias `SELECT`, `INSERT`, `UPDATE`, `DELETE`) se delega en componentes DAO que Spring Data implementa automáticamente en base a interfaces.

📌 6.1 Paginación Profesional: ¿Por qué Slice y no Page?

Para construir DAOs preparados para entornos empresariales, nuestras interfaces deben extender de la interfaz estándar `PagingAndSortingRepository<T, ID>` (la cual hereda todas las funciones CRUD tradicionales de `CrudRepository` y le añade capacidades nativas de ordenación y fraccionamiento).

⚠️ Regla de Escalabilidad de RAM en Servidores

Está terminantemente prohibido retornar colecciones masivas del tipo `List<Entity>` en métodos cuyas tablas tengan potencial de crecimiento indefinido en el tiempo (como históricos de pedidos). Cargar decenas de miles de filas simultáneamente en la memoria RAM degradará el recolector de basura de Java y terminará provocando un error de desbordamiento de memoria (`OutOfMemoryError`).

Para mitigar esto, definimos firmas que retornen porciones delimitadas de información parametrizadas con un objeto `Pageable`:

```
public interface OrderDao extends PagingAndSortingRepository<Order, Long> {
    Slice<Order> findByUserIdOrderByDateDesc(Long userId, Pageable pageable);
}
```

- **Pageable** : Parámetro que inyectamos en la llamada especificando de forma exacta qué índice de página requerimos y qué volumen máximo de elementos queremos descargar (ej. Página indexada en 0, tamaño de 15 registros).
- **Slice<T>** : Componente de retorno premium en arquitecturas modernas de alto rendimiento.

🔗 Justificación Arquitectónica de Examen: Slice vs. Page

Es muy habitual confundir la interfaz `Slice<T>` con `Page<T>`.

Si configuras el retorno como `Page<T>`, Spring Data se ve obligado a lanzar de forma transparente **una segunda consulta SQL oculta por debajo** del tipo `SELECT COUNT(*) FROM Tabla` para poder calcular el número total de páginas disponibles. En tablas masivas con millones de registros, este `COUNT(*)` degrada masivamente el rendimiento de la base de datos.

Al usar `Slice<T>`, Spring Data optimiza la consulta pidiendo simplemente un elemento extra en el `LIMIT` de SQL. De este modo, sabe si existe una página posterior para activar el flag `.hasNext()` **sin contar jamás la tabla completa**, maximizando la velocidad de respuesta.

📌 6.2 Consultas Declarativas y el uso de JPQL

- **Consultas por Convención de Nombres**: Spring Data analiza el nombre de tus métodos y genera automáticamente el árbol de ejecución SQL (ej. `findByFirstNameAndLastName` mapea a una cláusula `WHERE first_name = ? AND last_name = ?`). Si un método tiene la posibilidad de no hallar ningún registro, constituye una excelente práctica de diseño envolver el resultado en un contenedor `Optional<T>`.
- **JPQL (Java Persistence Query Language)**: Cuando las restricciones del buscador son complejas, hacemos uso de la anotación `@Query` implementando sentencias en lenguaje JPQL.

```
@Query("SELECT u FROM User u WHERE u.userName = ?1")
Optional<User> findUser(String userName);
```

🔗 Además de la sintaxis posicional (?1), JPQL soporta el uso de parámetros nombrados utilizando los dos puntos (:). Por ejemplo: `SELECT u FROM User u WHERE u.userName = :userName`. La API de JPA se encarga de sustituir `:userName` por el valor real en tiempo de ejecución.

🔗 Principio Fundamental de Examen: JPQL no es SQL

Grábate a fuego que **JPQL interroga exclusivamente a tus Clases Entidades de Java y a sus propiedades de objeto**, ignorando por completo el nombre físico de las tablas y columnas reales de tu base de datos SQL. Es decir, escribirás `FROM User u` (haciendo referencia a la clase `User`) en lugar de `FROM users_table` (nombre físico de la tabla). Hibernate se ocupará de su posterior traducción.

6.3 Consultas Dinámicas y el Patrón CustomizedDao

La anotación `@Query` no es válida cuando hay que generar dinámicamente la consulta (ej. búsqueda de productos por palabras clave y categoría que pueden ser nulas o variables). En estos casos, es necesario aplicar el patrón de extensión de DAOs en 3 pasos:

1. **Definir una interfaz propia:** Crear una interfaz adicional (ej. `CustomizedProductDao`) definiendo las firmas de búsqueda dinámica.
2. **Implementarla:** Crear la clase `CustomizedProductDaoImpl` (que implemente la interfaz anterior) haciendo uso de la API interna de JPA (`Criteria` o `EntityManager`) para construir la consulta dinámicamente en tiempo de ejecución.
3. **Herencia Múltiple:** Modificar la interfaz principal (`ProductDao`) para que extienda tanto de `CrudRepository` como de la nueva `CustomizedProductDao`. Spring las fusionará automáticamente.

7. Configuración de Entornos con YAML

Spring Boot permite orquestar los parámetros de conexión de persistencia mediante un fichero declarativo limpio denominado `application.yml`.

- **La Regla de Oro Sintáctica de YAML:** La jerarquía estructural del fichero se define de manera única y exclusiva haciendo uso de **espacios en blanco**. Está **estrictamente prohibido emplear tabuladores** (`\t`). Insertar un solo tabulador romperá el analizador sintáctico en tiempo de arranque e impedirá que el servidor inicie.
- **Aislamiento Absoluto de Entornos (Buenas Prácticas):** Para garantizar que la ejecución de pruebas automatizadas no contamine ni corrompa los almacenes de datos reales, segmentamos las configuraciones:
- *Entorno de Desarrollo (Perfil Default):* Direcciona a la base de datos principal (`pa`). Las tablas persisten en el tiempo y se alimentan de datos estables.
- *Entorno de Testing:* Al invocar la suite de pruebas (`mvn test`), Spring activa el perfil de test mediante `@ActiveProfiles("test")`, interceptando la configuración y conmutando las conexiones hacia una base de datos completamente aislada y limpia denominada `patest`.

8. Bloque de Evaluación Forense y Simulación de Examen

Resumen Rápido para Memorizar de Cara al Test

- JPA es la especificación ORM; Hibernate es la implementación ejecutora; JDBC es el canal subyacente de comunicación de bajo nivel.
- En relaciones bidireccionales 1:N, el **Lado Propietario** se sitúa en el extremo N, aloja la FK y se define mediante `@ManyToOne` junto a `@JoinColumn`. El **Lado Inverso** se ubica en el extremo 1, usa `@OneToMany` y exige el atributo `mappedBy`.
- La estrategia de **Acceso por propiedad** obliga a incrustar las anotaciones sobre los métodos *getter* de la clase.
- Los objetos de lógica de negocio puros dentro de las entidades se protegen con la anotación `@Transient`.
- Para optimizar la memoria y evitar queries ocultas del tipo `COUNT(*)`, los DAOs profesionales devuelven objetos parciales parametrizados mediante `Slice<T>` y gobernados por `Pageable`.

Análisis Forense de Caso Práctico (Pregunta 2 de Exámenes Históricos):

Disponemos de una arquitectura empresarial con una entidad principal `Order` vinculada en una relación 1:N hacia `OrderItem` configurada con estrategia de carga `LAZY`. A su vez, cada `OrderItem` mantiene una relación 1:1 hacia la entidad `Product`, también parametrizada como `LAZY`.

Se ejecuta de forma secuencial el siguiente fragmento de código dentro de una transacción válida:

```
Order order = orderDao.findById(id).get();           // Línea [A]
Set<OrderItem> items = order.getOrderItems();        // Línea [B]
for (OrderItem item : items) {                       // Línea [C]
    Product product = item.getProduct();             // Línea [D]
    System.out.println(product.getId());             // Línea [E]
    System.out.println(product.getName());           // Línea [F]
}
```

Pregunta del Examen:

¿Qué líneas concretas del script anterior provocan la ejecución de una **consulta de selección REAL (SELECT)** contra el motor de la base de datos relacional?

Dictamen Técnico Detallado Paso a Paso:

- **Línea [A] (SÍ causa SELECT):** Invoca de forma directa al componente DAO para recuperar un registro específico. El framework se ve obligado a lanzar una sentencia `SELECT` inmediata a la tabla de pedidos para instanciar y rellenar los datos del objeto `Order`.
- **Línea [B] (NO causa SELECT):** El método extrae la colección de ítems vinculados. Dado que la relación `@OneToMany` está configurada como `LAZY`, Hibernate **no acude a la base de datos**. En su lugar, retorna de forma instantánea una colección "falsa" rellena de proxies vacíos.
- **Línea [C] (SÍ causa SELECT):** El flujo del programa entra en el bucle `for` e intenta evaluar e iterar sobre los elementos de la colección. En este preciso milisegundo, la colección de proxies perezosa se ve obligada a activarse para poder suministrar los elementos reales. Hibernate interrumpe el hilo y **dispara una sentencia SELECT a la tabla `OrderItem`** para descargar todas las líneas de ese pedido.
- **Línea [D] (NO causa SELECT):** Se invoca el getter del producto desde el ítem actual. Como la relación es `LAZY`, el sistema inyecta de forma inmediata un objeto Proxy para representar al producto, esquivando la realización de cualquier query SQL.
- **Línea [E] (NO causa SELECT - ¡La Gran Trampa del Examen!):** El código solicita imprimir en consola el identificador del producto (`product.getId()`). El alumno descuidado asume que al pedir un dato del producto, el proxy `LAZY` se inicializará lanzando una query. **¡FALSO!** * *Explicación Avanzada:* Al encontrarnos inspeccionando un elemento `OrderItem` (que actúa como el **Lado Propietario** de la relación), la propia fila física de la tabla `OrderItem` en SQL **ya contiene de forma obligatoria la columna de la clave foránea `productId`**. Por lo tanto, cuando el proxy del producto fue creado en la línea anterior, Hibernate ya le inyectó el valor del ID que leyó del ítem. Como el proxy **ya conoce internamente su propio ID**, el método se resuelve en memoria RAM sin interactuar con la BD.
- **Línea [F] (SÍ causa SELECT):** El programa requiere el nombre del producto (`product.getName()`). El objeto proxy inspecciona su estructura interna y constata que carece de dicho atributo en caché. El hilo se congela y **se ejecuta una consulta SELECT real sobre la tabla de productos** para rellenar la entidad y poder extraer el string del nombre.

✓ Veredicto Oficial Concluyente

Las únicas líneas del fragmento analizado que causan una consulta física real (`SELECT`) contra la base de datos son de forma exclusiva: **la línea [A] (carga inicial), la línea [C] (apertura e iteración del bucle for) y la línea [F] (extracción de un atributo ordinario del proxy del producto).**

TEMA 3 - Capa de Lógica de Negocio con Spring

📌 Propósito de esta guía

Como tu profesor, te doy la bienvenida al "cerebro" de tu aplicación: **La Capa de Lógica de Negocio**. Aquí es donde las reglas de tu negocio (ej. *"no puedes comprar un carrito vacío"*, *"las transferencias no pueden dejar saldo negativo"*) se convierten en código.

Esta guía está diseñada para que domines los tres pilares de esta capa: **Inyección de Dependencias**, **Transaccionalidad (y estado dirty)**, y **Optimistic Locking**. Conocer esto te separará de un simple "picador de código" y te convertirá en un ingeniero de software, además de blindarte completamente contra las preguntas más capciosas del examen.

🗺 Índice del Tema

- [1. Inyección de Dependencias \(IoC Container\)](#)
- [2. Transaccionalidad Declarativa](#)
- [3. Optimistic Locking \(Control de Concurrencia\)](#)
- [4. Pruebas Automatizadas \(Testing\)](#)
- [5. Bloque de Repaso y Caso de Examen](#)

1. Inyección de Dependencias (IoC Container)

📌 1.1 Intuición y Concepto

Imagina que un servicio de tu aplicación (`ShoppingServiceImpl`) necesita acceder a la base de datos usando varios DAOs (`ProductDao` , `OrderDao`). En la programación tradicional, el servicio tendría que instanciar esos DAOs usando un patrón *Factory*. Esto acopla fuertemente el código y lo hace difícil de testear.

La solución moderna es la **Inversión de Control (IoC)** mediante Inyección de Dependencias.

📌 Intuición del IoC

Tú no buscas las herramientas que necesitas; un asistente (el Contenedor de Spring) te las pone en la mano antes de que empieces a trabajar.

📌 1.2 La Teoría Formal en Spring

Spring dispone de un **Contenedor de Beans**. Un *Bean* es simplemente un objeto de Java que Spring instancia, ensambla y gestiona por ti.

Fases de la Inyección:

1. **Declaración de Beans (El menú):** Le decimos a Spring qué clases queremos que convierta en Beans usando anotaciones a nivel de clase:
 - `@Service` : Para las clases que implementan Lógica de Negocio (ej. `UserServiceImpl`).
 - `@Repository` : Para los DAOs (Spring Data los implementa y registra automáticamente, es implícita).
2. **Inyección (El ensamblaje):** Usamos la anotación `@Autowired` sobre un atributo para pedirle a Spring: *"Búscame un Bean que implemente esta interfaz y asígnalo a esta variable"*.

```
@Service // 1. Le decimos a Spring: "Gestiona esta clase"
public class ShoppingServiceImpl implements ShoppingService {

    @Autowired // 2. Le decimos a Spring: "Inyecta la dependencia aquí"
    private ProductDao productDao;

    // ... lógica del servicio
}
```

🔗 Ámbito Singleton de los Beans:

Por defecto, el contenedor de objetos de Spring crea todos los beans (DAOs y Servicios) como **"singletons"**. Esto significa que solo existe **una única instancia** de cada DAO y servicio local compartida en toda la memoria del backend durante la ejecución de la aplicación.

2. Transaccionalidad Declarativa

Este es el concepto más crítico del tema y el que define el comportamiento seguro del backend.

📌 2.1 Intuición y Semántica Básica

Un caso de uso complejo (ej. `buy()`) implica múltiples lecturas y escrituras. Si ocurre un fallo a la mitad, no podemos dejar la base de datos en un estado inconsistente (ej. cobrado pero sin generar el pedido).

Todo el método debe ejecutarse como una **Transacción Atómica (todo o nada)**. En Spring, esto se logra anotando la clase o el método con `@Transactional`.

📌 2.2 Reglas de Ejecución y la Fórmula Matemática

Cuando se invoca un método anotado con `@Transactional`, Spring intercepta la llamada y aplica un flujo estricto:

- **Antes:** Si no hay transacción abierta, crea una nueva. Si ya existe una, se engancha a ella (propagación por defecto).
- **Durante:** Todas las operaciones de los DAOs invocados se asocian a esta misma transacción.
- **Después (Commit vs Rollback):** Sigue una regla estricta que debes grabar a fuego en tu memoria:

$$\text{Resultado} = \begin{cases} \text{COMMIT}, & \text{si finaliza bien } \vee E \in \text{CheckedException} \\ \text{ROLLBACK}, & \text{si } E \in \text{RuntimeException } \vee E \in \text{Error} \end{cases}$$

🔗 Trampa de Examen Frecuente

Es un error clásico creer que lanzar una excepción controlada (ej. `InstanceNotFoundException` o `EmptyShoppingCartException`) provoca un Rollback automático. **¡FALSO!** Las *CheckedExceptions* asumen un error de negocio previsible y hacen **COMMIT** de las operaciones exitosas previas. Para deshacer cambios de forma incondicional, la excepción debe ser *Unchecked* (`RuntimeException`).

📌 2.3 Optimizaciones: `readOnly = true`

Si un caso de uso es puramente de consulta (ej. `findOrder()`), debemos anotarlo con `@Transactional(readOnly=true)`.

- **Implicación:** Esto le dice al gestor de base de datos que no habrá escrituras, permitiéndole aplicar optimizaciones de rendimiento y bloquear menos recursos.

📌 2.4 Estado "Dirty" (¡Fundamental para entender Hibernate!)

Este comportamiento de JPA asusta a muchos principiantes, pero es brillante.

🔑 Concepto Clave: Estado Dirty

Si estás dentro de un método `@Transactional` y modificas una entidad cargada de la base de datos usando sus `setters`, **NO necesitas llamar a `dao.save()`**.

¿Por qué? Porque Hibernate vigila las entidades en memoria (Caché de primer nivel). Al terminar la transacción, detecta qué entidades han sido modificadas (*estado dirty*) y lanza automáticamente las sentencias `UPDATE` necesarias justo antes de hacer el Commit.

```
@Transactional
public User updateProfile(Long id, String name) {
    User user = userDao.findById(id).get(); // Se carga en caché
    user.setFirstName(name); // La Entidad pasa a estado "Dirty"

    // NO HAY userDao.save(user)
    return user;
} // <- Aquí Spring/Hibernate lanza el UPDATE automático en MySQL.
```

🔑 Parámetros de Entrada/Salida:

En los métodos de creación (como `userService.signUp(User user)`), el parámetro de la entidad actúa como un parámetro de **entrada/salida**. Tras su ejecución en la capa de persistencia, JPA muta el objeto original, por lo que `user.getId()` dejará de ser nulo y devolverá el identificador generado automáticamente por la BD.

📌 2.5 Caché de Primer Nivel

JPA mantiene una caché ligada estrictamente a la transacción actual. Si haces `userDao.findById(1)` y tres líneas después repites `userDao.findById(1)`, **solo se ejecuta un `SELECT` en MySQL**. La segunda vez se recupera instantáneamente de la memoria RAM. Al terminar la transacción, esta caché se destruye.

3. Optimistic Locking (Control de Concurrencia)

📌 3.1 El Problema: "Second Lost Update" (Actualización perdida)

Imagina dos empleados del banco (Transacción 1 y Transacción 2) que quieren añadir dinero a la cuenta 1 (saldo actual 1000€):

1. **T1** lee el saldo (1000€).
2. **T2** lee el saldo (1000€).
3. **T1** suma 100€ en memoria y guarda. BD = 1100€.
4. **T2** suma 200€ en memoria y guarda. BD = 1200€.

Resultado: Se han ingresado 300€ reales, pero la BD marca 1200€. ¡T2 sobrescribió el cambio de T1! Esto ocurre porque el nivel de aislamiento estándar (`READ_COMMITTED`) no bloquea las filas al leer.

⚠ Anomalía de Concurrency: El "Second Lost Update"

El nivel de aislamiento por defecto o estándar de las bases de datos (como `TRANSACTION_READ_COMMITTED`) no evita el problema conocido como **"second lost update"** (segunda actualización perdida).

Esto ocurre cuando dos transacciones (T1 y T2) leen una misma entidad (ej. un saldo de 1000€), la actualizan en memoria y realizan un *Commit*. Sin control, la T2 aplastaría silenciosamente los cambios de la T1 en la base de datos. La técnica de **Optimistic Locking** previene este fallo exigiendo un atributo anotado con `@Version`. Cuando JPA lanza el `UPDATE`, verifica que la versión en base de datos coincida con la que la transacción leyó inicialmente.

📌 3.2 La Solución: Optimistic Locking con @Version

En lugar de bloquear las tablas físicamente (*Pessimistic Locking*, que destruiría el rendimiento web), somos "optimistas" y asumimos que casi nunca habrá colisión. Si la hay, la detectamos al guardar y cancelamos.

Mecánica Formal:

1. Añadimos un atributo numérico a nuestra Entidad anotado con `@Version`.
2. Cuando Hibernate lanza el `UPDATE` automático (estado dirty), modifica la sentencia SQL para exigir que la versión en la BD coincida con la versión en memoria de Java, e incrementa la versión en 1:

```
UPDATE Account SET balance=1200, version=11 WHERE id=1 AND version=10
```

El colapso de la concurrency:

- **T1** guarda. El `UPDATE` funciona (la versión pasa a 11).
- **T2** intenta guardar. Lanza el `UPDATE ... WHERE id=1 AND version=10`.
- Como T1 ya cambió la versión a 11 en la base de datos, MySQL responde a T2 diciendo: *"0 filas afectadas"*.
- Hibernate ve "0 filas" y entra en pánico controlado: Lanza una `RuntimeException` (`OptimisticLockingFailureException`).
- La transacción de T2 hace un **Rollback automático** y se advierte al usuario.

4. Pruebas Automatizadas (Testing)

Probar un backend moderno requiere programar tests que levanten el contexto de Spring de manera aislada.

📌 4.1 Entorno y Anotaciones de Test

La clase de test (ej. `ShoppingServiceTest`) se decora con tres anotaciones vitales:

Anotación	Función y Comportamiento
<code>@SpringBootTest</code>	Levanta todo el contenedor de Spring, habilitando el uso de <code>@Autowired</code> en las pruebas.
<code>@ActiveProfiles("test")</code>	Carga <code>application.yml</code> y luego lo fusiona (no lo anula) con <code>application-test.yml</code> , sobrescribiendo propiedades como la URL de la BD aislada.
<code>@Transactional</code>	¡Cambio de semántica! En un test, asegura que la prueba se ejecute en transacción y, al acabar el método, SIEMPRE fuerza un Rollback .

💧 Consecuencia del Rollback en Tests

No tienes que limpiar la base de datos manualmente (`dao.delete(...)`) al final de cada test. El Rollback automático deja la base de datos inmaculada para el siguiente método de prueba.

📌 4.2 Igualdad de Entidades en Tests (El truco del equals)

¿Debemos sobrescribir los métodos `equals()` y `hashCode()` en nuestras Entidades para usar `assertEquals(user1, user2)` ?

- **Respuesta arquitectónica:** No es estrictamente necesario.
- **Justificación técnica:** Como cada test se ejecuta bajo una misma `@Transactional` , gracias a la **Caché de primer nivel**, si recuperas el mismo usuario dos veces, Hibernate devolverá la misma referencia exacta en memoria (mismo puntero). La igualdad referencial de Java (`==`) es suficiente.

5. Bloque de Repaso y Caso de Examen

📄 Resumen Rápido Blindado

La Lógica de Negocio agrupa reglas bajo `@Service` e inyecta DAOs con `@Autowired` . Protegemos las acciones compuestas agrupándolas atómicamente con `@Transactional` . La concurrencia se domestica de forma eficiente con Optimistic Locking (`@Version`), y todo se prueba aisladamente usando `@SpringBootTest` con rollbacks automáticos.

- **Estado Dirty:** Modificar entidades en una transacción implica un `UPDATE` invisible sin usar el DAO.
- **Caché L1:** Datos leídos se guardan en la RAM durante la transacción viva.
- **Regla Transaccional:** Checked Exception = `COMMIT` // Runtime Exception = `ROLLBACK` .

🔍 Mini Caso Práctico (Pregunta de Examen Real)

Enunciado:

Considere un servicio de la capa modelo, implementado con Spring y anotado con `@Transactional` . Valora las siguientes afirmaciones:

- **a)** Por defecto, si una operación lanza una "checked exception", termina en "rollback".
- **b)** Si se anota con `readOnly=true` , la operación NUNCA se ejecuta en transacción.
- **c)** Si se invoca otra operación de otro servicio también anotado con `@Transactional` , la segunda operación se engancha a la transacción de la primera.

Resolución Tutorizada Analítica:

- **Opción (a):** ❌ ¡FALSO! Ya hemos demostrado matemáticamente que las *checked exceptions* terminan en `COMMIT` por defecto.
- **Opción (b):** ❌ ¡FALSO! Sí hay transacción, pero en modo solo lectura, lo que instruye al driver JDBC para optimizar volcados de datos.
- **Opción (c):** ✅ ¡VERDADERA! Es el comportamiento clásico de la propagación transaccional por defecto (llamada `PROPAGATION_REQUIRED`). El segundo método verifica si ya hay una transacción abierta y, al existir una, se fusiona con ella, garantizando la atomicidad global del proceso.

TEMA 4 - Capa Servicios REST con Spring

📌 Propósito de esta guía

¡Bienvenido a la "Frontera" de tu aplicación! En los temas anteriores construimos el cerebro (Lógica) y la memoria (Datos) de nuestro backend. Ahora necesitamos una puerta de acceso para que el mundo exterior (el Frontend React o una App Móvil) pueda interactuar con él.

Como tu tutor, en este tema te enseñaré a construir Controladores REST robustos. Aprenderás a traducir peticiones HTTP de texto a objetos Java, a validar datos en la puerta de entrada, a manejar excepciones con elegancia, a hablar distintos idiomas (i18n) y a blindar tu aplicación con seguridad JWT.

🗺 Índice del Tema

- [1. El Ecosistema: Spring Boot y la "Magia" del Arranque](#)
- [2. Desarrollo de Controladores REST \(La Puerta de Entrada\)](#)
- [3. Validación Básica de Datos \(El Portero del Discoteca\)](#)
- [4. Gestión de Excepciones \(Traduciendo Errores a HTTP\)](#)
- [5. Internacionalización \(i18n\)](#)
- [6. Seguridad: Autenticación y Autorización \(JWT\)](#)
- [7. Bloque de Repaso y Análisis Forense de Exámenes](#)

1. El Ecosistema: Spring Boot y la "Magia" del Arranque

La Capa de Servicios REST se construye utilizando la librería *Spring Web*. Sin embargo, la orquestación global y el levantamiento de la aplicación se realiza a través de **Spring Boot**.

📌 1.1 Intuición y Ejecución

Spring Boot elimina el sufrimiento de configurar servidores web manualmente. Nuestra aplicación contará con una clase principal anotada con `@SpringBootApplication` que contiene un método `main`. Al ejecutar `SpringApplication.run(...)`, Spring levanta un **servidor web Tomcat embebido (interno)** y expone nuestros servicios al instante.

- **En Desarrollo (`mvn spring-boot:run`)**: Compila al vuelo, levanta el servidor embebido y escanea nuestras clases buscando anotaciones (`@RestController`, `@Service`, etc.) para instanciarlas en su Contenedor IoC.
- **En Producción (`mvn package`)**: Genera lo que conocemos en la industria como un **"Fat JAR"**.

🔥 Concepto de Examen: Fat JAR vs WAR

Un **Fat JAR** (*archivo .jar gordo*) incluye tu código Y el servidor web embebido junto con todas las dependencias. Se ejecuta directamente en cualquier máquina con el comando `java -jar mi-app.jar` sin necesidad de instalar nada más.

Si la arquitectura de tu empresa te obliga a desplegar sobre un Servidor de Aplicaciones tradicional externo (como un Tomcat standalone), tendrías que empaquetar tu código generando un archivo `.war`, no un Fat JAR.

2. Desarrollo de Controladores REST (La Puerta de Entrada)

Los controladores actúan como "receptionistas". Reciben una petición HTTP, extraen los datos, llaman al caso de uso de la Capa de Lógica de Negocio (nuestro `@Service`) y devuelven una respuesta estructurada en formato JSON.

📌 2.1 El Patrón DTO (Data Transfer Object)

¡Regla de Oro Arquitectónica! Nunca, bajo ninguna circunstancia, devuelvas tus Entidades JPA (Tema 2) directamente al exterior. ¿Por qué?

1. Expondrías la estructura interna y física de tu Base de Datos.
2. Podrías causar bucles infinitos al serializar a JSON si existen relaciones bidireccionales.
3. El mecanismo *LAZY Fetching* fallaría catastróficamente porque la sesión de persistencia y la transacción se cerraron al abandonar la Capa Lógica.

Solución: Usamos **DTOs**. Son clases Java simples (POJOs) que actúan como "sobres de correo" diseñados exclusivamente para viajar por la red. El controlador es el único responsable de convertir de Entidad a DTO antes de responder.

📌 2.2 Mapeo de Rutas y Parámetros

Anotamos nuestra clase con `@RestController` (lo que la convierte en un Bean de Spring e indica que todos sus métodos serializan su retorno a JSON automáticamente). Inyectamos nuestro servicio interno con `@Autowired` y mapeamos las rutas públicas:

$$\text{Mapeo HTTP} = \begin{cases} @GetMapping, & \text{Para leer/recuperar datos (Consultas sin alterar estado)} \\ @PostMapping, & \text{Para crear entidades o ejecutar acciones complejas (Login, Comprar)} \\ @PutMapping, & \text{Para actualizar/sobrescribir datos completos} \end{cases}$$

¿Cómo extraemos los datos que nos envía el cliente en la petición HTTP?

- **@PathVariable** : Extrae variables dinámicas incrustadas directamente en la ruta de la URL.
 - *Ruta:* `@GetMapping("/orders/{orderId}")`
 - *Firma:* `public OrderDto findOrder(@PathVariable Long orderId)`
- **@RequestParam** : Extrae parámetros adjuntos en la *query string* (`?keywords=matrix&page=0`).
 - *Opciones útiles:* Permite configurar `required=false` (lo hace opcional) o establecer un `defaultValue="0"`.
- **@RequestBody** : Extrae el cuerpo (*Body*) de la petición HTTP (codificado en JSON) y lo inyecta mágicamente transformándolo en un objeto de Java (DTO).
 - *Firma:* `public Long buy(..., @RequestBody BuyParamsDto params)`

🔗 Técnica Overloaded POST:

Para los casos de uso que no corresponden a operaciones CRUD estándar (como iniciar sesión en `/users/login` o registrarse en `/users/signUp`), la arquitectura rest dicta el uso de la técnica **Overloaded POST**. Se usa el método POST para poder transportar credenciales de forma segura en el cuerpo de la petición.

⚡ Trampa de Examen Frecuente

Si implementas un `@PostMapping` que espera recibir datos complejos (como un formulario de pago o credenciales de Login) en el cuerpo de la petición HTTP, tienes que usar la anotación `@RequestBody` obligatoriamente delante del parámetro en la firma del método. **Si lo olvidas**, Spring no sabrá dónde buscar el JSON, inyectará un objeto vacío o nulo, y la aplicación fallará.

3. Validación Básica de Datos (El Portero de Discoteca)

No debemos ensuciar nuestra Capa de Lógica de Negocio comprobando errores triviales (ej. que el usuario olvidó enviar su email o que el código postal tiene 50 caracteres). Delegamos la validación sintáctica a la puerta de entrada (Controlador) usando el estándar **Bean Validation**.

Mecánica de Validación Declarativa:

1. En nuestro DTO, configuramos las restricciones directamente sobre los atributos usando anotaciones como: `@NotNull`, `@Size(min=1, max=20)`, `@Min`, `@Email`.
2. En el Controlador, ordenamos a Spring que valide el DTO justo en el instante en el que lo inyecta añadiendo la anotación `@Validated` acompañando al `@RequestBody`.
 - *Ejemplo:* `public Long buy(..., @Validated @RequestBody BuyParamsDto params)`

Efecto Automático: Si el JSON recibido no cumple con las reglas del DTO, Spring aborta la ejecución antes de siquiera entrar a tu método y devuelve automáticamente un Error HTTP 400 (*Bad Request*).

🔗 `MethodArgumentNotValidException`:

Cuando la anotación `@Validated` falla porque un DTO incumple las reglas de validación de *Bean Validation*, Spring aborta la ejecución y lanza internamente la excepción de runtime

`MethodArgumentNotValidException`. Esta es la excepción exacta que nuestro `@ControllerAdvice` debe capturar con `@ExceptionHandler` para formatear los errores 400 Bad Request.

4. Gestión de Excepciones (Traduciendo Errores a HTTP)

Cuando nuestro "cerebro" (Capa Lógica) detecta un error de negocio semántico (ej. `EmptyShoppingCartException`), aborta el proceso lanzando una excepción nativa de Java. Dado que el frontend no comprende el *stacktrace* de Java, debemos traducir estas excepciones a **Códigos de Estado HTTP** comprensibles.

Tenemos dos estrategias de intercepción:

📌 4.1 Nivel Local (`@ExceptionHandler`)

Se emplea para gestionar excepciones que son muy específicas y exclusivas de un solo controlador. Se implementa creando un método de rescate dentro del propio controlador:

```
@ExceptionHandler({EmptyShoppingCartException.class})
@ResponseStatus(HttpStatus.NOT_FOUND) // Asigna un código HTTP 404
@ResponseBody // Fuerza la conversión del retorno a JSON
public ErrorsDto handleEmptyCart(EmptyShoppingCartException ex) {
    return new ErrorsDto("El carrito está vacío");
}
```

📌 4.2 Nivel Global (@ControllerAdvice)

Se emplea para capturar excepciones transversales a toda la aplicación (ej. `InstanceNotFoundException` generada por los DAOs, o los errores de validación desencadenados por `@Validated`).

Se implementa creando una clase separada y global anotada con `@ControllerAdvice`. Esta clase actúa como un radar, interceptando automáticamente los errores lanzados por cualquier controlador del proyecto y centralizando toda la lógica de respuesta a fallos.

📌 4.3 Extracción Programática de Errores de Validación (BindingResult)

En el nivel global (`@ControllerAdvice`), al interceptar una `MethodArgumentNotValidException` (el error HTTP 400 devuelto por `@Validated`), debemos extraer la información precisa para el cliente. La forma correcta de acceder a la lista de errores concretos de los campos del DTO es invocando programáticamente al método `exception.getBindingResult().getFieldErrors()`.

5. Internacionalización (i18n)

Cuando el backend debe enviar un mensaje de error legible al usuario, ¿en qué idioma debe redactarlo?

El proceso de internacionalización (i18n) en arquitecturas REST se gobierna mediante la **cabecera HTTP Accept-Language** que adjunta el navegador web (ej. `Accept-Language: es-ES`).

El Flujo de Traducción en Spring:

1. Definimos los diccionarios en ficheros de propiedades alojados en `src/main/resources`:
`messages_en.properties`, `messages_es.properties`.
2. Estos ficheros asocian pares clave-valor (ej. `project.exceptions.EmptyShoppingCartException=el carrito está vacío`).
3. Inyectamos un objeto gestor `MessageSource` en nuestro controlador de excepciones.
4. Los métodos `@ExceptionHandler` pueden declarar un parámetro automático de tipo `Locale`. Spring inyecta allí el idioma detectado en la cabecera HTTP de la petición entrante.
5. Invocamos `messageSource.getMessage("clave_del_error", variables, locale)` para recuperar y devolver el string correctamente traducido.

6. Seguridad: Autenticación y Autorización (JWT)

Nuestra API REST es *Stateless* (no tiene memoria ni estado). Entre una petición HTTP y otra, el servidor no recuerda quién eres. Para proteger áreas privadas sin usar sesiones de servidor que saturen la memoria, utilizamos **JSON Web Tokens (JWT)**.

- **Autenticación (Proceso de Login):** El usuario emite un POST a `/users/login` con sus credenciales. Si la capa lógica determina que son válidas, el backend cifra y empaqueta un "carnet VIP" (Token JWT) que contiene en su interior el ID del usuario y su Rol. Este token va firmado matemáticamente con un secreto que solo el servidor posee (Algoritmo HS256). El backend responde entregando este JWT al frontend.
- **Autorización (Consumo Privado):** Para cada petición a rutas protegidas, el frontend debe adjuntar obligatoriamente su JWT en una cabecera HTTP estándar llamada `Authorization` con el formato `Bearer <token>`.

Filtro Interceptor y el @RequestAttribute

En Spring Security, configuramos un Filtro personalizado (ej. `JwtFilter`) que intercepta la petición *antes* de que alcance el Controlador.

1. El filtro verifica la firma criptográfica del token (comprobando que nadie lo ha alterado).
2. Extrae el `userId` oculto en el cuerpo (*payload*) del token.
3. Inyecta ese `userId` verificado dentro de la petición interna de Spring mediante atributos.
4. **En el Controlador:** Añadimos `@RequestAttribute Long userId` en la firma de nuestro método REST.
¡Magia! Nuestro método inyecta un ID de usuario seguro y validado, impidiendo que un atacante suplante identidades manipulando parámetros de URL.

6.1 Control de Acceso Declarativo vs. Programático

- **Declarativo (Basado en Roles):** Spring Security permite definir políticas estáticas de acceso. Ej: `.requestMatchers(HttpMethod.POST, "/users/signUp").permitAll()` (Acceso Público), y bloquear por defecto todas las demás rutas obligando a estar autenticado.
- **Programático (Obligatorio en lógica semántica):** No todas las reglas de negocio pueden resolverse por el rol general del usuario. Para determinar si el *Pedido #14* le pertenece legítimamente al *Usuario #2*, el sistema de Roles es ciego. Necesitamos invocar a un componente especializado (`PermissionChecker`) de forma **programática** dentro de la Capa de Lógica.

6.1 Detalles Técnicos y Estrictos del Token JWT

El token JWT consta de 3 partes (Cabecera, Cuerpo y Firma) que están estrictamente codificadas en **base64url**. El cuerpo (*payload*) contiene información mediante *claims*. Además del identificador del usuario (`userId`) y su Rol (`role`), un token seguro incluye obligatoriamente la fecha de expiración mediante el claim `exp`, especificado en segundos transcurridos desde el 1-1-1970.

7. Bloque de Repaso y Análisis Forense de Exámenes

Resumen Rápido

La Capa REST orquesta la frontera exterior. Expone rutas con `@RestController`, inyecta cuerpos JSON en DTOs usando `@RequestBody`, ejecuta validaciones tempranas con `@Validated`, captura excepciones globales en `@ControllerAdvice` para devolver JSONs traducidos (i18n), y se blindo inyectando el ID del usuario con `@RequestAttribute` tras ser verificado por el filtro JWT de Spring Security.

- **DTOs:** Son los aislantes indispensables entre tu Base de Datos (Entidades) y el mundo exterior.
- **i18n:** El idioma se dictamina por la cabecera `Accept-Language`, gestionada internamente mediante el objeto `Locale`.

🔍 Mini Casos Prácticos (Análisis de Preguntas Oficiales Reales)

⚡ Trampa de Examen #1 (Ref. Pregunta 5 del PDF Oficial)

Enunciado: Analice el siguiente fragmento de código en la capa de Servicios REST correspondiente a transferencias de saldo:

```
@RestController
@RequestMapping("/accounts")
public class ShoppingController {
    @Autowired private AccountService accountService;
    @PostMapping("/transfer")
    public void transfer(..., @Validated TransferParamsDto params) throws ... {
        accountService.transfer(params.getSourceAccountId(), ...);
    }
}
```

Análisis y Resolución Tutorizada:

- ¿Falta `@Service` en lugar de `@Autowired`? → **FALSO**. `@Autowired` es la anotación correcta para solicitar la inyección de una dependencia. La anotación `@Service` se coloca en la definición de la clase de la lógica, no en el atributo que la recibe.
- ¿Los parámetros de la transferencia viajan en el *path* de la URL? → **FALSO**. Es una petición `POST` asociada a un DTO de parámetros complejos; estos datos viajan ocultos en el cuerpo (*Body*) de la petición, no en el enrutamiento.
- "Falta especificar la anotación `@RequestBody` sobre el parámetro `params`" → **¡VERDADERO!** Esta es la trampa clásica. Al recibir un JSON en el cuerpo de un POST, Spring es incapaz de mapearlo a un objeto Java a menos que le declares explícitamente la anotación `@RequestBody` delante del DTO.

⚡ Trampa de Examen #2 (Ref. Pregunta 10 del PDF Oficial)

Enunciado: Indique la afirmación correcta con respecto al desarrollo del backend bajo este enfoque arquitectónico.

Análisis y Resolución Tutorizada:

- ¿El JWT se adjunta como un parámetro en la URL de la forma `?authorization=token`? → **FALSO**. Las normativas de seguridad exigen enviar el token en la Cabecera HTTP `Authorization: Bearer <token>`.
- ¿Para desplegar en producción se requiere compilar un "Fat JAR" e instalarlo obligatoriamente en un servidor de aplicaciones externo? → **FALSO**. La gran ventaja del "Fat JAR" de Spring Boot es que lleva el servidor embebido (Tomcat) dentro de sí mismo y se auto-ejecuta de forma autónoma.
- ¿Los tokens JWT se envían por la red legibles en formato JSON puro? → **FALSO**. Los JWT viajan codificados en tres segmentos Base64url (cabecera.payload.firma).
- "No todo el control de acceso se puede resolver siempre declarativamente en base a roles" → **¡VERDADERO!** Aunque Spring Security bloquea el acceso general por roles (ej. aislar zona de Admins), autorizaciones semánticas dependientes de los datos (ej. comprobar que la factura que intentas descargar te pertenece) deben validarse obligatoriamente con código programático (el `PermissionChecker`).

TEMA 5 - Desarrollo Basado en Componentes con React

📌 Propósito de esta guía

¡Bienvenido al Frontend! Atrás dejamos las bases de datos y los servidores. Ahora entraremos en el navegador del usuario. El objetivo de este tema es cambiar tu forma de pensar: pasarás de ver páginas web estáticas (HTML tradicional) a construir **Interfaces de Usuario (IU) reactivas y modulares**.

Como tu profesor, te advierto: React tiene una curva de aprendizaje inicial peculiar debido a la inmutabilidad y al *Virtual DOM*. Esta guía está diseñada para que interiorices estos conceptos a la perfección y aniquiles las preguntas trampa del examen sobre cómo se gestiona el estado y cuándo se recarga la pantalla.

🗺 Índice del Tema

- [1. El Paradigma: Desarrollo Basado en Componentes](#)
- [2. Herramientas: Vite y el Entorno de Desarrollo](#)
- [3. El Estado de un Componente \(useState\)](#)
- [4. Propiedades \(Props\) y Listas](#)
- [5. El Virtual DOM \(El Motor de React\)](#)
- [6. Inmutabilidad \(La Regla de Oro\)](#)
- [7. Bloque de Repaso y Trampas de Examen](#)

1. El Paradigma: Desarrollo Basado en Componentes

📌 1.1 Intuición

Imagina la interfaz de Amazon. No es un documento monolítico gigante. Está formada por una barra de búsqueda superior, un menú lateral, listas de productos y tarjetas de producto individuales. El **Desarrollo Basado en Componentes** aborda una IU compleja tratándola como un puzzle de piezas de Lego.

- **¿Qué es un componente?** Es una pieza autónoma que genera un trozo de interfaz (HTML) y contiene la lógica para responder a lo que el usuario hace en ese trozo (clics, teclado).
- **Ventajas:** 1. *Divide y vencerás* (gestión de la complejidad).
2. *Reusabilidad* (si programas una excelente tarjeta de producto, puedes usarla en la vista de "Buscador" y en la vista de "Carrito").

📌 1.2 Introducción a React y JSX

React es la librería líder en la industria para materializar este paradigma. En React, un componente se programa como una **simple función de JavaScript** que devuelve el aspecto visual.

Para que escribir el aspecto visual sea cómodo, React inventó **JSX** (*JavaScript XML*). JSX te permite escribir código que parece HTML, pero que por debajo es 100% JavaScript.

```
const Welcome = () => {  
  return (  
    <div>  
      <h1>Welcome!</h1>  
    </div>  
  );  
};
```

⚡ Reglas de Oro de JSX (Trampas de Examen)

- **class vs className** : Como la palabra `class` está reservada en JavaScript para crear clases de programación, en JSX debes usar obligatoriamente `className` para asignar estilos CSS. (Pregunta oficial de examen).
- **Atributos camelCase**: Los eventos HTML cambian a camelCase. En lugar de `<button onClick="...">`, se escribe `<button onClick={...}>`.
- **Incrustar JavaScript**: Puedes meter variables de JS dentro del HTML usando llaves `{variable}`. **Nunca** uses `@variable` ni nada parecido. (Pregunta oficial de examen).

2. Herramientas: Vite y el Entorno de Desarrollo

Para no configurar transpiladores a mano (que traducen JSX a JS puro que el navegador entienda), usamos **Vite**.

- **En Desarrollo (`npm run dev`)**: Levanta un servidor ultrarrápido. Cada vez que guardas un cambio en tu código (`.jsx`), Vite lo intercepta, lo traduce al vuelo y recarga solo ese trozo en el navegador sin perder el estado de la aplicación.
- **En Producción (`npm run build`)**: "Exprime" tu código. Elimina comentarios, comprime (minimiza) todo el JavaScript y el CSS, y genera unos pocos ficheros listos para subirse a un servidor web real.
- **Ficheros `.jsx`** : Todo archivo que contenga sintaxis JSX (etiquetas HTML mezcladas con JS) debe llevar la extensión `.jsx`.

🔗 Compilación a Producción (Carpeta dist):

Al ejecutar el comando `npm run build` para producción, Vite minimiza todo el código JS y CSS y empaqueta el resultado dentro de una carpeta llamada **dist**. El contenido de esta carpeta es puramente estático y es el que se instala finalmente en el servidor web de producción.

3. El Estado de un Componente (`useState`)

Un componente estático es aburrido. Queremos que la interfaz reaccione y cambie.

📌 3.1 Intuición y Declaración

Si un componente necesita "recordar" algo entre clics (ej. el número de un contador o la lista de tareas), lo guarda en su **Estado**. Usamos un *Hook* llamado `useState`.

```
import { useState } from 'react';

const Counter = () => {
  // 1. Declaración usando destructuring
  const [value, setValue] = useState(0);

  // 2. Lógica
  const handleIncrement = () => setValue(value + 1);

  // 3. Renderización
  return (
    <div>
      <span>{value}</span>
      <button onClick={handleIncrement}>+</button>
    </div>
  );
};
```

📌 3.2 Cómo funciona `useState` matemáticamente

La función `useState(initialValue)` devuelve un array de dos posiciones:

$$\text{useState}(x_0) \rightarrow [\text{estadoActual}, \text{funcionParaActualizar}]$$

⚡ ¡TRAMPA MORTAL DE EXAMEN! (Pregunta 6 oficial)

Muchos alumnos ven la línea `const [todos, setTodos] = useState([]);` y piensan: "Si el componente se vuelve a renderizar, el array se vaciará y volverá a ser `[]`". **¡FALSO!**

El valor `[]` o `0` solo se asigna la primera vez que el componente se dibuja en pantalla (cuando se "monta").

En renderizaciones sucesivas provocadas por un `setX()`, React ignora ese parámetro inicial y devuelve el valor actual guardado en memoria.

📌 3.3 Reglas de los Hooks

Las funciones que empiezan por `use` (como `useState`, `useSelector`) son **Hooks**.

- Solo se pueden llamar en el **nivel superior** del componente.
- **No** puedes meter un Hook dentro de un `if`, un `for` o una función anidada. React depende del orden estricto de llamada para saber a qué variable de estado corresponde cada uno.

4. Propiedades (Props) y Listas

Los componentes no están aislados; se comunican de arriba hacia abajo pasando datos a los que llamamos **Propiedades (props)**.

📌 4.1 Paso de Props

El componente padre `App` pasa la lista de tareas al componente hijo `Todos` así: `<Todos todos={miLista} />`.

El hijo la recibe siempre en un objeto. Por limpieza, aplicamos *destructuring* directamente en los parámetros:

```
const { todos } = props // Extrae 'todos' del objeto props
```

📌 4.2 La directiva mágica `key` (Alerta de rendimiento)

Cuando vas a iterar un array para dibujar una lista de elementos (usando el método `.map()` de JS), React te obliga a ponerle a cada elemento una propiedad especial llamada `key`.

```
{todos.map(todo => (  
  <Todo key={todo.id} todo={todo} />  
))}
```

¿Por qué? React necesita saber exactamente qué elemento de la lista se ha añadido, borrado o modificado. Si le pasas un identificador único (como el ID de la base de datos) en la `key`, React puede actualizar la lista en el navegador en milisegundos. Si no lo pones, el rendimiento se desploma y saltará un *warning*.

5. El Virtual DOM (El Motor de React)

¿Por qué se llama React y por qué es tan rápido?

📌 5.1 El Problema Clásico

Manipular el DOM del navegador directamente (hacer `document.getElementById('caja').innerHTML = ...`) es la operación más lenta y costosa de la programación web moderna.

📌 5.2 La Solución: El Virtual DOM

React utiliza un truco brillante. Mantiene una copia exacta, pero ligera y en memoria RAM, de la interfaz. A esto se le llama **Virtual DOM**.

🔗 Analogía Intuitiva: El Juego de las 7 Diferencias

1. Cada vez que tú llamas a una función `setAlgo()`, React ejecuta la función de tu componente entero y genera un nuevo HTML.
2. En lugar de aplastar el navegador con este nuevo HTML, lo guarda en el Virtual DOM (es instantáneo porque es RAM pura).
3. React pone el Virtual DOM "Viejo" junto al Virtual DOM "Nuevo" y juega a las diferencias (**Algoritmo de Diffing**).
4. Descubre que: "¡Ajá! La única diferencia es que la etiqueta `<li>` número 3 ha cambiado a 'completed'".
5. React va al navegador real y modifica **ESTRICTAMENTE** ese atributo. Nada más.

🔗 Pregunta Fija de Examen

- ¿Los componentes React se renderizan en el DOM del navegador o en el Virtual DOM?
- **Respuesta Oficial:** Se renderizan en el **Virtual DOM**. Luego, React extrae las diferencias y aplica los parches mínimos al DOM real del navegador. (Pregunta 8 del PDF oficial).

6. Inmutabilidad (La Regla de Oro)

Para que el juego de "Las 7 Diferencias" (Virtual DOM) sea veloz, React necesita saber rápido si una variable de estado ha cambiado o no.

📌 6.1 Mutar vs Crear

❌ **Enfoque incorrecto (Mutación):**

```
todos.push(nuevaTarea); // Modificamos el array original (el puntero es el mismo)
setTodos(todos);        // ¡Peligro!
```

Si haces esto, React compara el puntero en memoria del array antiguo con el nuevo. Como es exactamente la misma zona de memoria, `todos === todos` da true y React deduce: "Oh, es el mismo objeto, no me molesto en re-renderizar nada". Tu interfaz se queda congelada.

✅ **Enfoque correcto (Inmutabilidad con Spread Operator):**

```
const nuevosTodos = [nuevaTarea, ...todos]; // Clonamos y añadimos
setTodos(nuevosTodos);
```

Creamos un nuevo array en una zona de memoria distinta. Al comparar, React ve que el puntero ha cambiado (`todos !== nuevosTodos`), sabe que hubo una actualización y dibuja los cambios.

6.2 Optimizaciones basadas en Inmutabilidad

Tratar el estado como inmutable permite optimizaciones agresivas. React solo necesita comparar si la referencia de memoria (el puntero) cambió mediante una comprobación superficial (*Shallow comparison*, usando `===`). Es una operación de microsegundos frente a iterar miles de elementos uno a uno.

7. Bloque de Repaso y Trampas de Examen

Resumen Rápido

React divide la IU en componentes que devuelven JSX. El estado se guarda con `useState`. Cuando llamas a la función `set` (pasando siempre un objeto NUEVO, respetando la **inmutabilidad**), React vuelve a ejecutar el componente en su **Virtual DOM**. Busca las diferencias exactas y actualiza el DOM del navegador de forma ultra-eficiente.

Casos Prácticos (Análisis de Exámenes Oficiales Reales)

Cuestión Oficial (Ref. Pregunta 6)

Dado el código: `const [todos, setTodos] = useState([]);`
`const handleAddTodo = text => setTodos([todo(text), ...todos]);`

- ¿En cada renderización la variable `todos` se reinicia a `[]`? → **FALSO**. `useState` recuerda el valor real entre renderizaciones.
- ¿La ejecución de `handleAddTodo` causará una re-renderización? → **VERDADERO**. Cualquier invocación a la función `set` empuja a React a re-dibujar el componente.

Cuestión Oficial (Ref. Pregunta 11)

Sobre la sintaxis en tu frontend:

- a) En JSX hay atributos de tags HTML que no usan su nombre original (ej. `className` en vez de `class`).
- b) En JSX se usan variables con `@variable`.

Respuesta correcta: A. `className` es vital. Las variables se inyectan con `{variable}`, nunca con `@`.

Cuestión Oficial (Ref. Pregunta 8)

Indique la afirmación correcta:

- a) Los componentes React se renderizan en el Virtual DOM.
- c) Se renderizan en el DOM del navegador directamente...

Respuesta correcta: A. Siempre en el Virtual DOM, que actúa de amortiguador de rendimiento.

TEMA 6 - Gestión de Estado con Redux

📌 Propósito de esta guía

¡Bienvenido al sistema nervioso central de tu Frontend! Cuando la aplicación crece, pasar datos de componente en componente se vuelve insostenible. Esta guía te enseñará a dominar **Redux**, el estándar de la industria para la gestión de estado global.

Entenderás su flujo unidireccional, la magia matemática detrás de su rendimiento ($===$) y, crucialmente, cómo estructurar los datos como una base de datos real (**Estado Normalizado**) para lograr una eficiencia algorítmica perfecta de $\mathcal{O}(1)$.

🗺 Índice del Tema

- [1. Motivación: El Problema del Estado Compartido](#)
- [2. Conceptos Principales de Redux](#)
- [3. Integración de Redux en React \(React-Redux\)](#)
- [4. Arquitectura Avanzada de Reductores](#)
- [5. Bloque de Repaso y Análisis de Examen](#)

1. Motivación: El Problema del Estado Compartido

🔪 1.1 El infierno de pasar propiedades ("Prop Drilling")

En una aplicación sencilla, el componente raíz (`App`) guarda todo el estado y lo pasa hacia abajo mediante *props*. En una aplicación real, esto es inviable.

- **Caos de mantenimiento:** Componentes intermedios reciben propiedades que no usan, solo para pasarlas mecánicamente a sus hijos.
- **Colapso del Virtual DOM:** Cualquier cambio en el estado del componente raíz obliga a re-renderizar todo el árbol hacia abajo de manera ineficiente.

🔪 1.2 La ilusión de los "Componentes Contenedores"

Podríamos intentar distribuir el estado en "componentes contenedores" intermedios. Sin embargo, en aplicaciones empresariales como *PA-Shop*, los datos se cruzan por ramas del árbol completamente separadas:

- **Ejemplo del Usuario:** El componente de `Login` inicializa los datos; el `Header` necesita el nombre para mostrarlo arriba; la vista `UpdateProfile` los edita abajo.
- **Ejemplo del Carrito:** `ProductDetails` añade ítems; `ShoppingCartIcon` (en la cabecera) muestra el total acumulado.

Conclusión Arquitectónica: No hay un "padre común" cercano que pueda alojar el estado de manera elegante. Necesitamos extraer el estado del árbol de React y colocarlo en un **"Almacén Global" (Store)**.

🔗 Componentes de Presentación:

El diseño de interfaces complejas a menudo separa las responsabilidades entre componentes *contenedores* (que gestionan estado o lógica compleja) y **Componentes de Presentación**. Estos últimos son componentes pasivos que simplemente reciben parte del estado como propiedades y *callbacks* para manipular la interfaz ante acciones del usuario.

2. Conceptos Principales de Redux

Redux se fundamenta en tres pilares que siguen un **flujo de datos estrictamente unidireccional**.

📌 2.1 El store (El Almacén)

Es el único objeto que contiene todo el estado global de la aplicación. Es un árbol de objetos de **solo lectura**.

📌 2.2 Las Acciones (Actions)

Son la única forma permitida para solicitar un cambio en el *store*.

- Son objetos planos de JavaScript.
- **Requisito Obligatorio:** Deben tener una propiedad `type` (ej. `type: 'ADD_TODO'`).
- Se envían al sistema mediante la función `store.dispatch(action)`.

📌 2.3 El Reductor (Reducer)

Es el "cerebro" matemático que transforma el estado evaluando la acción.

$$\text{Reducer}(\text{state}_t, \text{action}) \rightarrow \text{state}_{t+1}$$

⚡ Reglas de Oro del Reductor (Pregunta Fija de Examen)

1. **Es una Función Pura:** Para los mismos parámetros de entrada, siempre devuelve el mismo resultado. Está prohibido ejecutar llamadas asíncronas a APIs o generar efectos secundarios dentro de él.
2. **Inmutabilidad Absoluta:** ¡NUNCA modifica el objeto `state` que recibe como parámetro! Debe crear y devolver un nuevo objeto clonando los datos anteriores usando el *spread operator* (`...state`).

3. Integración de Redux en React (React-Redux)

Para conectar el ecosistema de React con el motor de Redux, usamos la librería `react-redux` y `@reduxjs/toolkit`.

📌 3.1 Suscripción al Estado: El Hook `useSelector`

Los componentes leen el *store* usando `useSelector(selector)`. Un selector es una función en línea que extrae exclusivamente el dato necesario del gran árbol (ej. `state => state.todos`).

🔮 La Magia Matemática del Rendimiento (===)

¿Cómo sabe React si debe volver a renderizar un componente tras recibir una acción? Redux ejecuta todos los selectores suscritos y compara el resultado nuevo con el resultado antiguo usando **igualdad referencial estricta (===)**.

- Si mutaste el estado (*rompiendo las reglas del reductor*), el puntero de memoria es el mismo (`true`), y la pantalla **no se actualiza**.
- Si respetaste la inmutabilidad devolviendo un objeto nuevo, el puntero cambia (`false`), alertando a React para que aplique el parche en pantalla.

🔮 API Nativa de Redux:

Aunque en React usamos los hooks `useSelector` y `useDispatch` proporcionados por la librería `react-redux`, a nivel subyacente la API pura de Redux expone los métodos `store.subscribe()` para escuchar cambios y `store.getState()` para leer el estado del árbol global en cualquier momento.

📌 3.2 Disparo de Acciones: El Hook `useDispatch`

Permite a los componentes funcionales enviar acciones al *store*.

```
const dispatch = useDispatch();
// ...
<button onClick={() => dispatch(actions.increment())}> + </button>
```

4. Arquitectura Avanzada de Reductores

📌 4.1 Creadores de Acciones (*Action Creators*)

Son funciones auxiliares que generan los objetos de acción para evitar errores tipográficos y parametrizar la información de envío (*payload*).

```
export const addTodo = text => ({ type: 'ADD_TODO', todo: {text} });
```

📌 4.2 El Idioma del Estado Inicial

Al arrancar la aplicación por primera vez, Redux envía una acción vacía para inicializar todo el árbol. El reductor debe estar preparado usando argumentos por defecto (`state = initialState`) y contar de forma obligatoria con un caso `default: return state` en su `switch` para responder adecuadamente sin romper la app.

📌 4.3 Divide y Vencerás: `combineReducers`

En aplicaciones reales, el estado se trocea. Usamos `combineReducers` para unir pequeños "mini-reductores" independientes (uno gestiona a los usuarios, otro el catálogo de productos) en un solo gran Reductor Raíz monolítico.

📌 4.4 El Estado Normalizado (Eficiencia y Escalabilidad)

⚡ Concepto Crítico de Arquitectura (Nivel Avanzado)

Guardar colecciones de datos como simples Arrays nativos en Redux (ej. `state.todos = [{id: 1}, {id: 2}]`) genera un grave problema de rendimiento en aplicaciones grandes.

El Problema del Array ($\mathcal{O}(n)$):

Si necesitas modificar la tarea con `id: 5000` o verificar si existe, el reductor se ve obligado a ejecutar `.map()` o `.find()` , iterando elemento por elemento. A nivel de complejidad algorítmica, esto asume un **coste computacional lineal** $\mathcal{O}(n)$.

La Solución Estándar: Estado Normalizado ($\mathcal{O}(1)$):

Redux exige estructurar el estado emulando una base de datos relacional, separando radicalmente los datos físicos del orden semántico de los mismos usando diccionarios (*Objetos JS*).

- **byId (El Diccionario de Entidades):** Un objeto indexado donde las claves son los IDs unívocos y los valores son las propias entidades. Acceder, leer o modificar el elemento 5000 es instantáneo llamando a `state.byId[5000]` . **Coste computacional constante** $\mathcal{O}(1)$.
- **allIds (El Array de Orden):** Un simple array plano de identificadores `["1", "2"]` que dicta a React en qué orden estricto deben dibujarse los elementos en la pantalla.

Ejemplo Visual de Estado Normalizado:

```
const state = {
  todos: {
    byId: {
      "1": { id: 1, text: "Aprender React", completed: false },
      "2": { id: 2, text: "Aprender Redux", completed: true }
    },
    allIds: ["1", "2"]
  }
}
```

Si actualizamos un elemento, ya no iteramos el array entero; simplemente clonamos el objeto contenedor y sobrescribimos la clave específica en el diccionario.

5. Bloque de Repaso y Análisis de Examen

Resumen Rápido

Redux centraliza el estado global. Los componentes se suscriben con `useSelector` (optimizado bajo la validación de punteros `===`) y solicitan cambios disparando acciones con `useDispatch`. Los reducers procesan estas intenciones mediante **funciones puras e inmutables**. Para garantizar un rendimiento extremo a gran escala, dividimos el estado con `combineReducers` y **normalizamos** las colecciones transformándolas en diccionarios `byId`.

- **Inmutabilidad:** Modificar el estado localmente (`state.value = 5`) rompe la alerta de actualización de la pantalla de React.
- **Estado Normalizado:** Diccionario (`byId`) garantiza búsquedas instantáneas $\mathcal{O}(1)$ y Array paralelo (`allIds`) gestiona el orden de iteración visual.

🔥 Casos Prácticos (Análisis Forense de Exámenes Oficiales)

⚠️ Pregunta de Examen Real: Redux y Renderizaciones (Ref. Pregunta 7)

Enunciado: Tienes una aplicación con un icono (`UnreadMessagesIcon`) que muestra un globo rojo con el número total de mensajes sin leer. Cada vez que borras permanentemente un mensaje (que ya había sido leído previamente), el array global `messages` cambia de longitud.

- **Trampa 1:** ¿El reductor debe buscar el mensaje en el array, modificar internamente su atributo y devolver el mismo array? → **¡FALSO!**. Eso destruiría la regla de inmutabilidad absoluta del reductor.
- **Trampa 2:** ¿El componente `UnreadMessagesIcon` se volverá a renderizar al borrar un mensaje ya leído dado que el array físico ha cambiado? → **¡FALSO!**. El `useSelector` de este icono concreto devuelve un número entero (ej. `5` mensajes sin leer). Al borrar de la base de datos un mensaje que **ya estaba leído**, el selector de Redux sigue calculando y devolviendo `5`. Como internamente detecta que `5 === 5` es `true`, React omite por completo la renderización del icono mágicamente.
- **Respuesta Correcta del Examen:** La única certeza es que *la ejecución del `dispatch` provoca siempre la ejecución íntegra del reductor raíz*, independientemente de si la interfaz se re-dibuja o no.

⚠️ Pregunta de Examen Real: El Virtual DOM (Ref. Pregunta 8)

Indique la afirmación correcta sobre las renderizaciones:

- **Trampa:** Redux reduce el número de renderizaciones en el DOM del navegador. → **¡FALSO!**. React siempre amortigua las interacciones directas con el navegador. Lo que Redux permite es reducir dramáticamente las renderizaciones inútiles en el **Virtual DOM** de aquellos componentes que no están afectados por un cambio de estado global.
 - **Respuesta Correcta:** *"Los componentes React se renderizan en el Virtual DOM"*. Constituye la premisa arquitectónica de todo el framework estudiado.
-

TEMA 7 - Caso de Estudio (Frontend de la tienda PA-Shop)

📌 Propósito de esta Guía Maestra

¡Bienvenido a la culminación de tu aprendizaje en el Frontend! Como tu profesor, he diseñado esta Versión 3.0 Definitiva combinando la estructura oficial más estricta del temario con la máxima profundidad técnica y pedagógica posible.

Aquí vamos a ensamblar todas las piezas (React, Redux, React Router, Vite) utilizando la aplicación PA-Shop como hilo conductor. Recuerda la regla de oro de la asignatura: **No necesitas memorizar el código de PA-Shop**. Lo que el examen evaluará con dureza es si entiendes la arquitectura, los patrones, y el porqué de cada decisión de diseño para que sepas aplicarlas a cualquier software empresarial en el futuro. ¡Vamos a asegurar esa Matrícula de Honor!

🗺 Índice del Tema

- [7.1 Arquitectura Global](#)
- [7.2 Caso de Uso "Buscar Productos"](#)
- [7.3 Caso de Uso "Ver Detalles de un Producto"](#)
- [7.4 Caso de Uso "Comprar"](#)
- [7.5 Otros Aspectos \(Arquitectura Web Fundamental\)](#)
- 🧑‍🔬 [Análisis Forense de las Trampas Oficiales](#)

7.1 Arquitectura Global

En lugar de crear un monolito inmanejable metiendo todos los componentes en una sola carpeta, PA-Shop utiliza un enfoque profesional **multi-modular**.

📌 El Patrón de Fachada (index.js) y Reductor Raíz

Cada módulo funcional (ej. `users`, `catalog`, `shopping`) actúa como una "mini-aplicación" que agrupa casos de uso relacionados, encapsulando sus propios componentes, acciones y su mini-reductor.

Para evitar un acoplamiento tóxico entre módulos, empleamos el **Patrón de Fachada**. Cada módulo tiene un fichero `index.js` en su raíz que define estrictamente qué se exporta al exterior (es una convención arquitectónica vital). Si el módulo `shopping` necesita mostrar el catálogo, no rebusca en las carpetas internas de `catalog`; simplemente importa de su puerta principal:

```
import { FindProducts } from '../..catalog';
```

Finalmente, el reductor raíz (`rootReducer.js`) fusiona todos los mini-reductores usando `combineReducers` para construir el gran árbol de estado global de Redux.

📌 Layout, Enrutamiento e i18n

- **React Router:** Las *Single Page Applications* (SPA) nunca recargan la página completa de forma física. Para simular la navegación, envolvemos la app en `<BrowserRouter>`. El componente `App` define el *layout* (Header, Body, Footer). En el `Body`, usamos `<Routes>` y `<Route>` para dictar qué componente se pinta según la URL. Además, usamos `<Link to="...">` en lugar de la clásica etiqueta HTML `<a href="...">` para interceptar el clic y cambiar el Virtual DOM sin realizar peticiones web de refresco al servidor.

- **Internacionalización (i18n):** Usamos *React Intl*. El idioma preferido viaja en la cabecera HTTP `Accept-Language`. Un componente `<IntlProvider>` inyecta los diccionarios (archivos de mensajes) a toda la app. Sustituimos los textos estáticos por componentes dedicados como `<FormattedMessage id="clave.diccionario" />`.

🔗 Componentes de Formateo de Tiempo:

Además de `<FormattedMessage>` y `<FormattedNumber>`, la librería *React Intl* proporciona los componentes `FormattedDate` y `FormattedTime`. Estos son imprescindibles para renderizar correctamente fechas como las de los pedidos (`order.date`) respetando el *locale* del usuario.

`ForwardedDate` y `ForwardedTime`

📌 La Abstracción Crítica: `appFetch`

🔗 Concepto de Examen (Comunicaciones HTTP)

En la Capa de Acceso a Servicios nunca usamos el `fetch` nativo de JavaScript directamente desperdigado por los componentes. En su lugar, construimos una función envoltorio (*wrapper*) centralizada llamada `appFetch`.

¿Por qué es vital usar `appFetch` en la arquitectura?

1. **Comodidad:** Construye automáticamente la URL final añadiendo la ruta base del backend (gestionando los entornos de desarrollo vs producción vía variables de Vite).
2. **Seguridad Automática:** Extrae el token JWT del almacenamiento del navegador e inyecta de forma automatizada la cabecera HTTP `Authorization: Bearer <token>` en todas las peticiones privadas si el usuario está autenticado.
3. **Procesamiento Inteligente:** Devuelve un objeto limpio `{ ok, payload }` donde `ok` es `true` si el código HTTP es de la familia 2xx, y `payload` contiene el JSON ya desempquetado. Adicionalmente, centraliza la gestión de errores de red mostrando ventanas modales globales si el servidor está caído.

🔗 Codificación de URLs (`encodeURIComponent`):

Al realizar peticiones asíncronas de búsqueda, si los parámetros de la cadena de consulta (Query String) contienen caracteres especiales o espacios (como en `keywords`), es obligatorio procesarlos envolviéndolos con la función nativa `encodeURIComponent(keywords)` para evitar romper la sintaxis de la petición HTTP.

7.2 Caso de Uso "Buscar Productos"

Este caso de uso es fundamental para entender cómo los componentes interactúan con la entrada del usuario y el estado global.

Controlled vs. Uncontrolled Components

Existen dos paradigmas arquitectónicos para leer datos de un `<input>` en React. Esta es una pregunta clásica de examen:

Paradigma	Ubicación de los Datos	Mecánica de Lectura	Recomendación / Control
Uncontrolled (Estilo antiguo / "Pull")	Los datos viven exclusivamente en el DOM real del navegador.	Usas el atributo especial <code>ref</code> para capturar el nodo HTML y, cuando se ejecuta el evento, "tiras" (<i>pull</i>) del dato leyendo <code>input.value</code> .	No recomendado. React permanece "ciego" ante lo que hace el usuario hasta el último instante.
Controlled (Estándar React / "Push")	Los datos viven en el Estado Local (<code>useState</code>) del componente.	El atributo <code>value</code> del input se ata estrictamente a la variable de estado (<code>value={name}</code>), y el evento <code>onChange</code> actualiza el estado con cada pulsación.	El estándar en PA-Shop. Permite interceptar, validar y transformar datos en tiempo real (Push).

Analogía Intuitiva: El Muñeco Ventrílocuo

Un Input Controlado es como un muñeco ventrílocuo. No tiene voluntad propia ni puede mostrar texto por sí mismo. Cuando el usuario teclea "A", el input avisa a React (`onChange`). React guarda "A" en su estado. Luego, React le dice al input: "Oye, tu *value* ahora es A". Esto nos permite interceptar datos en tiempo real; por ejemplo, forzar que todo se escriba en MAYÚSCULAS transformando el dato en el `onChange` antes de guardarlo en el estado.

El Estado Global en Búsquedas

Cuando el usuario pulsa "Buscar", disparamos una acción asíncrona que guarda en Redux tanto los criterios de filtrado (`categoryId`, `keywords`, `page`) como los resultados obtenidos (`items`).

Gracias a este diseño, si el usuario navega para ver los detalles de un producto específico y luego pulsa el botón "Atrás" del navegador, el componente `FindProductsResult` simplemente lee la última búsqueda guardada en Redux (vía `useSelector`) y **dibuja la tabla al instante sin tener que volver a realizar una consulta HTTP al servidor**.

7.3 Caso de Uso "Ver Detalles de un Producto"

Parámetros de Ruta (`useParams`)

Al hacer clic en una tarjeta de producto, el enrutador navega hacia la ruta estandarizada `/catalog/product-details/:id`. Para extraer ese identificador variable del *path* (ej. extraer el número 14) y saber qué producto solicitarle al backend, usamos el Hook `useParams()` de React Router, el cual nos devuelve un objeto mapeado con los parámetros de la URL.

📌 El Hook `useEffect` y la Función de Limpieza (*Clean-up*)

⚡ Atención: Funcionamiento Avanzado del Ciclo de Vida

En React, el cuerpo de un componente debe ser una función puramente matemática que devuelve JSX. Si necesitas comunicarte con el mundo exterior (un efecto secundario como hacer un `appFetch` al backend), debes hacerlo obligatoriamente dentro de un `useEffect`.

La firma sintáctica del hook es:

```
useEffect(funciónEfecto, [dependencias])
```

El comportamiento del ciclo de vida depende estrictamente de la parametrización de su array de dependencias:

Sintaxis del Array	Frecuencia de Ejecución	Riesgos / Comportamiento
Sin array <code>useEffect(() => {...})</code>	Se ejecuta después de cada una de las renderizaciones del componente.	Alto peligro de provocar un bucle infinito si se altera el estado en su interior.
Array vacío <code>useEffect(() => {...}, [])</code>	Se ejecuta solo una vez , inmediatamente después de que el componente se monta en pantalla.	Ideal para peticiones de carga inicial única (<i>fetch</i> de datos).
Array con variables <code>useEffect(() => {...}, [id])</code>	Se ejecuta al montar, y posteriormente solo si el valor de la variable <code>id</code> cambia.	Optimiza el rendimiento de red reaccionando de forma selectiva a cambios de parámetros.

El Flujo de Ejecución en `ProductDetails` :

1. El componente se monta inicialmente vacío (el estado `product` es nulo).
2. Se ejecuta el `useEffect`, lanzando la petición de red asíncrona al servidor a través de `appFetch`.
3. Llega el JSON con los datos, se invoca a `setProduct()`, lo que altera el estado local y obliga a React a re-renderizar la pantalla mostrando finalmente los datos del producto.

📌 La Función de Limpieza (*Clean-up*)

Opcionalmente, la función de tu efecto puede retornar otra función. A esta última se la conoce formalmente como **función de limpieza**.

- **¿Para qué sirve?** Es una pieza de ingeniería vital. Si el usuario entra a los detalles de un producto y rápidamente cierra la pantalla o navega a otra sección antes de que el servidor responda, la petición de red se queda "volando". El *clean-up* permite **cancelar peticiones HTTP activas, destruir temporizadores (`clearInterval`) o desuscribirse de eventos globales de teclado**, previniendo graves fugas de memoria (*memory leaks*) en el navegador.
- **¿Cuándo se ejecuta?** React la invoca automáticamente justo antes de que el componente se destruya (desmonte), o inmediatamente antes de volver a ejecutar el efecto si las variables del array de dependencias han cambiado.

🔗 Nota de Experto: El comportamiento de StrictMode

Si usas React 18 con `StrictMode` activado en desarrollo, observarás que el componente se monta → ejecuta el *clean-up* → y se monta de nuevo de forma instantánea. Esto provoca que veas dos peticiones de red seguidas en la consola. No es un bug de tu código; React lo hace a propósito para obligarte a comprobar que tu función de limpieza está perfectamente programada y no rompe el estado.

7.4 Caso de Uso "Comprar"

En este flujo nos enfrentamos a la necesidad crítica de validar la información de pago y dirección de envío antes de sobrecargar al backend con peticiones erróneas.

📌 Validaciones del Lado Cliente (Programáticas)

Si delegamos la validación en el comportamiento automático estándar del navegador, este mostrará ventanas flotantes ("globos") de alerta nativos muy difíciles de personalizar. Para una aplicación profesional, tomamos el control absoluto:

1. Añadimos el atributo `noValidate` a la etiqueta `<form>` de HTML. Esto le ordena al navegador: *"Apaga tus globos automáticos de error estructural, yo gobierno la validación"*.
2. Cuando el usuario hace click en enviar (*submit*), invocamos programáticamente la API de validación nativa de HTML5 mediante el método `form.checkValidity()`.
3. Si la validación falla (ej. correo mal formateado, campo obligatorio vacío), usamos componentes de control de layout como `Form.Control.Feedback` de React Bootstrap para renderizar nuestros propios mensajes de error, perfectamente integrados en el diseño y los colores de la aplicación.

📌 Gestión de Errores Devueltos por el Backend

¿Qué ocurre si la validación del lado cliente es superada con éxito, pero el servidor backend rechaza la compra (Error HTTP 400 *Bad Request*) indicando que las reglas semánticas del DTO se han roto (ej. carrito vacío)?

🔗 Arquitectura de Errores (Aviso Crítico de Examen)

Los errores devueltos por un formulario desde el backend **NUNCA ENSUCIAN EL ESTADO GLOBAL DE REDUX**.

Al tratarse de errores efímeros y volátiles (como *"El formato del código postal es incorrecto"*), que solo le interesan a la pantalla que el usuario está operando en ese preciso milisegundo, se almacenan estrictamente en el **Estado Local** del componente usando un Hook: `const [backendErrors, setBackendErrors] = useState(null)`. Un componente específico `<Errors>` lee esta variable local y dibuja las alertas rojas en pantalla.

7.5 Otros Aspectos (Arquitectura Web Fundamental)

📌 Recargas (F5) y Supervivencia del Estado

Durante el ciclo de uso normal, un usuario puede pulsar la tecla F5 para recargar la página de forma manual.

- **El Problema:** Esta acción destruye por completo el entorno de ejecución de JavaScript en la pestaña. Toda la memoria RAM se limpia. Consecuencia: **¡El árbol global de Redux desaparece por completo!** El usuario perdería su sesión y su carrito al instante.
- **La Solución (`sessionStorage`):** En el momento en que se completa la autenticación exitosa, guardamos el token JWT en el `sessionStorage` del navegador. Esta API de almacenamiento sobrevive a recargas manuales (F5) pero se destruye automáticamente si el usuario cierra la pestaña por seguridad.

- **Mecánica del Login Silencioso:** Al volver a arrancar tras el F5, el componente raíz `App` inspecciona el `sessionStorage`. Si halla un token JWT activo, dispara de forma silenciosa e invisible la ruta del backend `/users/loginFromServiceToken`. El servidor valida el token y retorna el perfil completo del usuario junto a su carrito, **reconstruyendo el estado global de Redux de forma mágica y transparente** sin que el cliente note interrupción ni deba reintroducir sus credenciales.

🔒 Política Same-Origin y CORS

- **El escenario:** Nuestro Frontend de Vite se sirve típicamente desde el origen `http://localhost:5173`. Nuestro Backend de Spring ejecuta sus servicios en `http://localhost:8080`.
- **El bloqueo:** Los navegadores web implementan de forma estricta la **Política del Mismo Origen** (*Same-Origin Policy*), la cual prohíbe por defecto que un script descargado desde un origen (puerto 5173) realice peticiones asíncronas (`fetch`) hacia un origen o dominio diferente (puerto 8080), con el fin de evitar ataques de inyección de código cruzado (*Cross-Site Scripting*).
- **La solución:** El backend de Spring debe configurarse explícitamente para emitir cabeceras **CORS** (*Cross-Origin Resource Sharing*) en sus respuestas HTTP. Estas cabeceras actúan como una autorización, notificándole al navegador: *"Confío plenamente en las peticiones procedentes del puerto 5173, permito que lea mis recursos"*.

7.6 Bloque de Evaluación y Resumen

📄 Resumen Rápido a Memorizar

- **7.1 Arquitectura:** Usa `index.js` como fachada, un reductor raíz, y centraliza las peticiones con `appFetch` para inyectar automáticamente los tokens JWT y lidiar con JSON.
- **7.2 Formularios:** Usa componentes controlados atando el `value` al estado de React.
- **7.3 Ciclo de vida:** El `useEffect` permite lanzar peticiones al montar, pero debe retornar una función de *clean-up* para evitar fugas de memoria.
- **7.4 Validación:** Se usa `noValidate` en el HTML y `checkValidity()` programáticamente. Los errores de validación del backend van al estado local, jamás a Redux.
- **7.5 Red:** El `sessionStorage` salva al JWT de las recargas F5. Las cabeceras CORS permiten a puertos distintos comunicarse rompiendo el *same-origin*.

🕵️ Análisis Forense de las Trampas Oficiales

🔗 Análisis de la "Ruta Comodín" (`/*`)

Si examinas el archivo de enrutamiento `Body.jsx`, observarás una ruta definida de la siguiente forma:

```
<Route path="/*" element={<Home/>}/>.
```

- **La trampa del test:** ¿Qué pasa si la URL introducida en el navegador es `/catalog/find-products`? ¿Se renderizará el componente `Home` dado que el asterisco comodín acepta cualquier cadena de caracteres?
- **La respuesta arquitectónica: NO.** El componente `<Routes>` de React Router implementa un motor de búsqueda inteligente que evalúa todas las rutas declaradas y **elige siempre la ruta que posea la coincidencia más específica**. Como la ruta exacta de catálogo existe y está declarada, renderiza el catálogo. El sistema solo caerá en la ruta comodín (`/*`, que actúa como un cajón de sastre de error) si el usuario introduce una URL rota o inexistente, usándose para redirigir al inicio o pintar una página 404.

⚡ Análisis de la Pregunta Oficial de Examen (Ref. Pregunta 12)

Enunciado: Indique la afirmación correcta con respecto al enfoque de desarrollo estudiado en la asignatura para el desarrollo del frontend:

- **a)** "En general, los errores devueltos por una petición de formulario al backend se guardan en el estado global de Redux." → **¡FALSO!** Como analizamos en la sección 7.4, esto ensucia el árbol global de forma innecesaria con datos volátiles. Se almacenan en el estado local del componente mediante `useState`.
 - **b)** "Las restricciones HTML5 se ejecutan de forma automática al pulsar submit sin que el desarrollador tenga que escribir código." → **¡FALSO!** En nuestra arquitectura desactivamos explícitamente los globos nativos introduciendo `noValidate={true}` en el `<form>` y programamos la validación manual invocando a `checkValidity()` dentro del evento `onSubmit`.
 - **c)** "Un componente frontend no debe especificar restricciones HTML5 si estas validaciones ya se realizan de forma segura en el backend." → **¡FALSO!** La validación debe ser siempre **bidireccional**. En el frontend sirve para proporcionar feedback instantáneo al usuario (mejorando la experiencia) y ahorrar tráfico de red y computación innecesaria al servidor. En el backend actúa como la última línea de defensa infalible contra ataques.
 - **d)** Ninguna de las anteriores. → **¡RESPUESTA CORRECTA!** Esta pregunta discrimina con dureza a los alumnos que programan memorizando código mecánicamente sin comprender el flujo de control de errores de la arquitectura de la asignatura.
-

TEMA 8 - Arquitecturas y Tecnologías Relacionadas

Propósito de esta guía

¡Felicidades, has llegado al final del temario! A lo largo de los temas anteriores hemos construido *PA-Shop* como una aplicación completa pero centralizada. En este último tema, nos ponemos el "**sombrero de Arquitecto de Software**".

Vamos a analizar cómo las aplicaciones empresariales reales evolucionan para soportar millones de usuarios simultáneos, pasando del modelo monolítico a los Microservicios y explorando el salto tecnológico del entorno web clásico al ecosistema móvil nativo.

Aviso Pedagógico: Al ser el material fuente original un índice esquemático, he expandido rigurosamente las explicaciones con estándares consolidados de la industria (como Docker, AWS o patrones distributivos) para garantizarte un nivel premium de preparación ante el examen.

Índice del Tema

- [1. Evolución del Backend: Del Monolito a los Microservicios](#)
- [2. El Frontend: Más allá de la Web \(Mobile & Desktop\)](#)
- [3. Aplicabilidad Práctica de los Conocimientos Adquiridos](#)
- [4. Bloque de Repaso y Simulación de Entrevista](#)

1. Evolución del Backend: Del Monolito a los Microservicios

1.1 La Arquitectura Monolítica (El Modelo Clásico)

Hasta ahora, hemos diseñado la aplicación bajo una **arquitectura monolítica**. En este paradigma, todos los controladores (`CatalogController` , `UserController`), los servicios de lógica de negocio y los DAOs de persistencia coexisten dentro de la **misma base de código (un solo proyecto)** y se ejecutan bajo un único proceso unificado en el servidor.

- **Ventaja:** Sencillo de desarrollar, testear localmente y desplegar cuando el equipo de ingenieros y el alcance del proyecto son acotados.
- **Desventaja de Escalabilidad:** Si durante un evento de rebajas el componente de catálogo recibe millones de impactos pero el de registro de usuarios no sufre variaciones, estás obligado a clonar/escalar **la aplicación monolítica entera**, desperdiciando recursos del servidor. Además, un fallo fatal en un hilo de cualquier componente puede corromper la memoria y tumbar el sistema completo.

1.2 Arquitectura Basada en Microservicios (μS)

Para resolver las limitaciones de concurrencia y aislamiento de fallos, fragmentamos el monolito en servicios especializados e independientes que se ejecutan en procesos aislados y se comunican a través de llamadas de red (usualmente protocolos REST/JSON o mensajería asíncrona):

- `\mu S$ Catalog` (Gestión del catálogo de productos).
- `\mu S$ User` (Gestión de perfiles y autenticación).
- `\mu S$ ShoppingCart` (Gestión del estado de los carritos).
- `\mu S$ Order / Buy` (Procesamiento de pedidos y pasarela de pago).

🔗 Taxonomía de Microservicios:

Analizando la arquitectura desplegada, los microservicios se dividen visual y arquitectónicamente en dos roles: los **Microservicios Core** (como `Catalog`, `User`, `Order`, que acceden directamente a una BD independiente) y los **Microservicios de Composición/Orquestación** (como `Buy`, que no tiene BD propia y orquesta lógicamente las peticiones hacia el Carrito y el Pedido).

🔗 Regla de Oro Ineludible de los Microservicios (¡Pregunta de Examen!)

En una arquitectura de microservicios pura **NO existe una única base de datos central compartida**. Cada microservicio gestiona y es dueño exclusivo de su propio almacenamiento relacional o documental (`BD \mu S User`, `BD \mu S Catalog`, etc.).

Esto garantiza que si la base de datos de usuarios colapsa por sobrecarga, el catálogo web siga operativo y sirviendo datos. Sin embargo, traslada la complejidad hacia la gestión de transacciones distribuidas y la consistencia eventual.

📌 1.3 El API Gateway (La Pasarela de Entrada)

Si el cliente frontend tuviera que memorizar e invocar la dirección IP y el puerto de cada uno de los microservicios distribuidos por la red, el acoplamiento sería inmanejable.

Para solucionar esto, se interpone el componente **API Gateway**. Actúa como el **único punto de entrada público**. El frontend (React) emite todas sus peticiones HTTP directamente al Gateway, y este se encarga de enrutar de forma inteligente el tráfico hacia el microservicio correspondiente en la red interna privada.

📌 1.4 Infraestructura de Despliegue: Contenedores y Cloud

Componente	Definición Tecnológica	Propósito Arquitectónico
Contenedores	Abstracción ligera que empaqueta una aplicación y todo su entorno de ejecución (librerías, binarios, variables) en un bloque estandarizado e independiente (ej. <i>Docker</i>).	Garantiza que el código funcione exactamente igual en el ordenador del desarrollador que en producción, aislando los procesos. Es mucho más ligero que una máquina virtual al no replicar el S.O. completo.
Escalabilidad Horizontal	Multiplicación instantánea de instancias de un contenedor.	Permite replicar de forma automatizada aquellos microservicios con picos de carga (ej. levantar 20 contenedores de <code>Catalog</code> y dejar solo 2 de <code>User</code>), aportando tolerancia a fallos (si un contenedor cae, otro lo reemplaza).
Cloud Provider	Infraestructura física y lógica bajo demanda en la nube (ej. <i>AWS</i> , <i>Google Cloud</i> , <i>Azure</i>).	Proporciona el cómputo, red y almacenamiento global sobre el cual se despliega y orquesta el ecosistema de contenedores.

2. El Frontend: Más allá de la Web (Mobile & Desktop)

El diseño arquitectónico basado en componentes y la separación de responsabilidades que hemos ejercitado a lo largo del curso faculta a las tecnologías web para colonizar otros entornos nativos de ejecución.

2.1 Multiplataforma desde un Único Código Fuente

Tradicionalmente, desarrollar para entornos nativos requería duplicar o triplicar esfuerzos: programar en *Swift* para iOS, en *Kotlin* para Android y en *JavaScript* para la Web. El enfoque moderno unifica el desarrollo:

- **React Native:** Emplea exactamente la misma sintaxis JSX, las reglas de inmutabilidad, los Hooks (`useState` , `useEffect`) y el almacén global de Redux aprendidos en el curso. La diferencia radical radica en que, en lugar de renderizar los parches sobre un Virtual DOM que inyecta etiquetas HTML (`<div>` , `<span>`), el framework opera sobre un puente tecnológico que dibuja **componentes nativos reales del sistema operativo** de *iOS* (`UIView`) y *Android*.
- **Electron / Tauri (Escritorio Nativo):** Permiten empaquetar tu aplicación web SPA (*Single Page Application*) junto a un motor de renderizado ligero para distribuirla como una aplicación de escritorio nativa instalable en *Windows*, *Linux* o *macOS* (ej. herramientas líderes como *VS Code*, *Slack* o *Discord* implementan esta arquitectura exacta).

3. Aplicabilidad Práctica de los Conocimientos Adquiridos

Todo el esfuerzo invertido en las unidades previas constituye el pilar fundamental de las arquitecturas avanzadas del mercado de desarrollo de software:

1. **Spring Boot (Backend):** Es el rey indiscutible de la industria para codificar los microservicios descritos en la sección 1. Cada microservicio paquetizado de forma aislada dentro de un contenedor será, en esencia, un **Fat JAR** de Spring Boot gobernado por su fichero de propiedades YAML y exponiendo contratos de comunicación vía API REST.
 2. **React y Redux (Frontend):** Dominar la inmutabilidad de variables, el flujo unidireccional de datos, los selectores de rendimiento y la asincronía en las llamadas de red no se limita a navegadores web; compone el set de habilidades exacto y más demandado por grandes compañías para construir aplicaciones móviles nativas escalables a través de *React Native*.
-

4. Bloque de Repaso y Simulación de Entrevista

Resumen Rápido Blindado

Las aplicaciones masivas migran de la **Arquitectura Monolítica** hacia los **Microservicios**. En este modelo, el backend se descompone en mini-aplicaciones independientes, cada una dueña absoluta de su propia base de datos. Se ejecutan aisladas en **Contenedores** dentro de un **Cloud Provider** y se exponen al exterior mediante un **API Gateway** unificado. El frontend capitaliza el ecosistema de componentes de React para desplegar de forma nativa en móviles (*React Native*) y escritorio (*Electron*).

Mini Caso Práctico (Simulación de Entrevista Técnica de Arquitectura)

Entrevistador: "Tenemos una aplicación web monolítica desarrollada en Spring Boot para un comercio electrónico. Durante las campañas de rebajas, experimentamos caídas catastróficas por cuellos de botella en la base de datos centralizada. Queremos migrar el sistema hacia una arquitectura de microservicios basada en contenedores en la nube. ¿Cómo estructurarías los datos y la gestión del tráfico de entrada?"

Respuesta Ideal Recomendada (Sobresaliente):

"Para resolver este escenario, descompondría inicialmente el monolito dividiéndolo en dominios de negocio claramente delimitados, transformándolos en microservicios independientes: un servicio para el Catálogo, otro para Usuarios y otro para las Compras.

Como decisión arquitectónica crítica, **asignaría una base de datos totalmente independiente y dedicada a cada microservicio**, erradicando así el punto único de fallo y el cuello de botella de la base de datos centralizada.

Posteriormente, empaquetaría cada microservicio dentro de **Contenedores independientes**. Esto nos otorgará la capacidad de realizar una **escalabilidad horizontal elástica en la nube**, replicando de forma masiva (múltiples instancias) únicamente el contenedor del Catálogo y de Compras durante las rebajas, optimizando el coste de infraestructura.

Finalmente, para evitar que los clientes web y móviles tengan que gestionar las direcciones de red de toda esta malla interna, dispondría un **API Gateway** en la frontera como único punto de entrada público. El Gateway interceptará todo el tráfico del frontend, validará la seguridad y se encargará de enrutar eficientemente cada petición HTTP hacia el microservicio interno correspondiente."